“Alexandru Ioan Cuza” University of Iași
Faculty of Mathematics

Dimensionality Reduction Techniques for Machine Learning. The UMAP Algorithm

Master’s Dissertation

Scientific Supervisor:
Assoc. Prof. Dr. Iulian Stoleriu

Student:
Vlad Ioan Ciocîrlan

July, 2025
Iași

Contents

1 Introduction **3**
1.1 Probability in Multidimensional Balls . . . 4
1.2 Relevance of the Observation and Gaussian Distribution . . . 4
1.3 The Curse of Dimensionality and Solutions . . . 5

2 SNE Algorithm **7**
2.1 Introduction to SNE . . . 7
2.2 Computing the p_{ij} probabilities . . . 9
2.3 Optimization in SNE . . . 13

3 Symmetric SNE Algorithm **17**
3.1 Differences between SNE and Symmetric SNE . . . 17
3.2 Optimization in Symmetric SNE . . . 19

4 tSNE Algorithm **21**
4.1 The Crowding Problem and tSNE . . . 21
4.2 Optimization in tSNE . . . 24

5 UMAP Algorithm **26**
5.1 UMAP and Generalized Cross-Entropy . . . 26
5.2 UMAP and Nearest Neighbor Descent . . . 27
5.3 UMAP and Binary Search . . . 28
5.4 UMAP and Laplacian Eigenmaps . . . 29
5.5 UMAP and Non-Linear Least Squares . . . 34
5.6 UMAP and Stochastic Gradient Descent . . . 35

A Appendix – Python Codes **39**
Appendix A – Python Code . . . 39
A.1 SNE . . . 39
A.1.1 Computing p_{ij} probabilities for SNE . . . 39
A.1.2 Optimization in SNE . . . 40
A.1.3 Testing SNE . . . 42
A.2 Symmetric SNE . . . 43
A.2.1 Computing p_{ij} probabilities in Symmetric SNE . . . 43
A.2.2 Optimization in Symmetric SNE . . . 44
A.2.3 Testing Symmetric SNE . . . 45
A.3 tSNE . . . 45
A.3.1 Computing p_{ij} and q_{ij} probabilities in tSNE . . . 45

A.3.2 Optimization in tSNE 47
A.3.3 Testing tSNE 48
A.4 UMAP 48
A.4.1 UMAP and Nearest Neighbors Descent 48
A.4.2 UMAP and Binary Search 48
A.4.3 UMAP and Laplacian Eigenmaps 50
A.4.4 UMAP and Non-Linear Least Squares 51
A.4.5 UMAP and Stochastic Gradient Descent 51
A.4.6 Testing UMAP 52

Chapter 1

Introduction

Let Ω be a set of objects and $X : \Omega \to \mathbb{R}^n$ be a **random feature vector** of these objects. In data science, we have a finite subset of Ω for which we know the values of X; and, based on this information (which we call **training data**), we want to approximate various properties of the distribution of vector X or correlations with other random variables. A problem that arises in this procedure is called **the curse of dimensionality**, which we will explain in this introductory chapter.

Let $||.|| : \mathbb{R}^n \to [0, \infty)$ be a norm. We denote $B(a, r) = \{x \in \mathbb{R}^n | ||x - a|| < r\}$ as the ball with center $a \in \mathbb{R}^n$ and radius $r \in (0, \infty)$ with respect to this norm. If we have two such balls $B(a_1, r_1), B(a_2, r_2)$, we can write the following relation:

$$B(a_2, r_2) = a_2 + B(0, r_2) = a_2 + \frac{r_2}{r_1} B(0, r_1) = a_2 + \frac{r_2}{r_1}(B(a_1, r_1) - a_1) = \frac{r_2}{r_1} B(a_1, r_1) + \left(a_2 - \frac{r_2}{r_1} a_1\right),$$

where the absolute homogeneity of the norm justifies the possibility of factoring out scalars in these relations. Defining the function $f : \mathbb{R}^n \to \mathbb{R}^n, f(x) = \frac{r_2}{r_1} x + \left(a_2 - \frac{r_2}{r_1} a_1\right)$, we obtain that $B(a_2, r_2) = f(B(a_1, r_1))$. Let $\lambda : \mathcal{M} \to [0, \infty)$ be the **Lebesgue measure** defined on $\mathcal{M} \subset \mathcal{P}(\mathbb{R}^n)$, the set of all Lebesgue measurable subsets of $\mathbb{R}^n$. Any ball is Lebesgue measurable because it is an open set, and thus a Borel set. The choice of norm does not matter in defining the ball here, as any two norms on a finite-dimensional real linear space are equivalent, meaning they induce the same topology; hence, open sets with respect to one norm are also open with respect to any other norm. Thus, it follows directly from the definition of the Lebesgue integral that $\lambda(B(a_2, r_2)) = \int_{B(a_2, r_2)} 1 d\lambda = \int_{f(B(a_1, r_1))} 1 d\lambda$. Furthermore, since the function f is linear, it is clearly a C^1-diffeomorphism (because its inverse is also linear, thus C^1-smooth), the set $B(a_1, r_1)$ is open, and the constant function 1 is Lebesgue measurable and positive; we can therefore apply the **change of variables theorem**:

$$\int_{f(B(a_1, r_1))} 1 d\lambda = \int_{B(a_1, r_1)} |\det J_f| d\lambda,$$

where J_f denotes the Jacobian of the function f. From the two equalities in the preceding sentences, we obtain by transitivity that $\lambda(B(a_2, r_2)) = \int_{B(a_1, r_1)} |\det J_f| d\lambda$. Next, we observe that $\forall i, j \in \overline{1, n}, \frac{\partial f_i}{\partial x_j} = \frac{r_2}{r_1} \mathbf{1}_{i=j} \Rightarrow J_f = \frac{r_2}{r_1} I_n \Rightarrow \det J_f = \left(\frac{r_2}{r_1}\right)^n > 0 \Rightarrow |\det J_f| = \left(\frac{r_2}{r_1}\right)^n$. Thus, we deduce that $\lambda(B(a_2, r_2)) = \int_{B(a_1, r_1)} \left(\frac{r_2}{r_1}\right)^n d\lambda = \left(\frac{r_2}{r_1}\right)^n \int_{B(a_1, r_1)} 1 d\lambda = \left(\frac{r_2}{r_1}\right)^n \lambda(B(a_1, r_1))$.

Since any two norms in $\mathbb{R}^n$ are equivalent, the norm $||.||$ used in this reasoning is also equivalent to the L^∞ norm, so $B(a_1, r_1)$ includes a ball in the L^∞ norm, which is a cube. This implies that $\lambda(B(a_1, r_1))$ is greater than or equal to the Lebesgue measure of a cube, which is strictly positive by definition. Hence, since $\lambda(B(a_1, r_1)) > 0$, we can deduce that $\lambda(B(a_2, r_2)) = \left(\frac{r_2}{r_1}\right)^n \lambda(B(a_1, r_1)) \Rightarrow \frac{\lambda(B(a_2,r_2))}{\lambda(B(a_1,r_1))} = \left(\frac{r_2}{r_1}\right)^n$.

In the previous paragraph, "ball" referred to an open ball. We denote $\overline{B(a,r)} = \{x \in \mathbb{R}^n | ||x - a|| \leq r\}$ as the closed ball with center $a \in \mathbb{R}^n$ and radius $r \in (0, \infty)$. Let $\epsilon \in (0, \infty)$. Clearly, we have

$$B(a,r) \subset \overline{B(a,r)} \subset B(a, r+\epsilon) \Rightarrow \lambda(B(a,r)) \leq \lambda(\overline{B(a,r)}) \leq \lambda(B(a, r+\epsilon))$$

$$\Rightarrow 1 \leq \frac{\lambda(\overline{B(a,r)})}{\lambda(B(a,r))} \leq \frac{\lambda(B(a,r+\epsilon))}{\lambda(B(a,r))} = \left(\frac{r+\epsilon}{r}\right)^n.$$

Since this result holds for any $\epsilon > 0$, we deduce that

$$1 \leq \frac{\lambda(\overline{B(a,r)})}{\lambda(B(a,r))} \leq \lim_{\epsilon \searrow 0} \left(\frac{r+\epsilon}{r}\right)^n = \left(\frac{r+0}{r}\right)^n = 1^n = 1$$

$$\Rightarrow \frac{\lambda(\overline{B(a,r)})}{\lambda(B(a,r))} = 1 \Rightarrow \lambda(\overline{B(a,r)}) = \lambda(B(a,r)).$$

Thus, we have obtained that the Lebesgue measure of a closed ball coincides with the Lebesgue measure of the corresponding open ball. Therefore, the formula demonstrated in the previous paragraph remains valid for closed balls as well: $\frac{\lambda(\overline{B(a_2,r_2)})}{\lambda(\overline{B(a_1,r_1)})} = \left(\frac{r_2}{r_1}\right)^n$. In the remainder of this chapter, "ball" will refer to a closed ball.

1.1 Probability in Multidimensional Balls

Next, over Ω, we assume the existence of a **probability space** $(\Omega, \mathcal{F}, P)$, and we further assume that X is absolutely continuous, and its **probability density function** $p_X : \mathbb{R}^n \to [0, \infty)$ is continuous (which is true, at least locally, for most continuous distributions encountered in practice). We also consider a ball $\overline{B(a, r_1)}$ that includes a concentric ball $\overline{B(a, r_2)}$, implying $r_2 < r_1$. Since X is absolutely continuous, and the ball $\overline{B(a, r_1)}$ is a closed set, and thus Borel measurable, we have that $P(X \in \overline{B(a, r_1)}) = \int_{\overline{B(a,r_1)}} p_X(x)dx$ exists. Since $\overline{B(a, r_1)}$ is a compact and connected set, and p_X is a continuous function, it follows from the **Mean Value Theorem for integrals** that there exists an $x_1 \in \overline{B(a, r_1)}$ such that $P(X \in \overline{B(a, r_1)}) = \int_{\overline{B(a,r_1)}} p_X(x)dx = p_X(x_1)\lambda(\overline{B(a, r_1)})$. Similarly, it can be shown that there exists an $x_2 \in \overline{B(a, r_2)}$ such that $P(X \in \overline{B(a, r_2)}) = p_X(x_2)\lambda(\overline{B(a, r_2)})$. If we assume that $P(X \in \overline{B(a, r_1)}) \neq 0$ (a plausible assumption if the ball $\overline{B(a, r_1)}$ includes the training data), we can divide the relations to obtain

$$\frac{P(X \in \overline{B(a, r_2)})}{P(X \in \overline{B(a, r_1)})} = \frac{p_X(x_2)}{p_X(x_1)} \cdot \frac{\lambda(\overline{B(a, r_2)})}{\lambda(\overline{B(a, r_1)})} = \frac{p_X(x_2)}{p_X(x_1)} \cdot \left(\frac{r_2}{r_1}\right)^n.$$

1.2 Relevance of the Observation and Gaussian Distribution

Now, let's explain the relevance of the equality obtained above. If we can bound the ratio $\frac{p_X(x_2)}{p_X(x_1)}$ by a constant, then we obtain that $\frac{P(X \in \overline{B(a,r_2)})}{P(X \in \overline{B(a,r_1)})}$ decreases exponentially to 0 as n increases,

because $r_2 < r_1 \Rightarrow \frac{r_2}{r_1} < 1$. Such a property is evident for a uniform distribution, but we will also demonstrate it for a **Gaussian distribution**, as this is one of the most frequently encountered distributions.

So, let's assume that $X \sim \mathcal{N}(\mu, \Sigma)$, which means

$$p_X(x) = \frac{1}{(2\pi)^{\frac{n}{2}}(\det \Sigma)^{\frac{1}{2}}} \mathrm{e}^{-\frac{1}{2}\langle \Sigma^{-1}(x-\mu), x-\mu\rangle},$$

where $\mu \in \mathbb{R}^n$ is the mean of the random vector X, and $\Sigma \in \mathcal{M}_n(\mathbb{R})$ is its covariance matrix. Thus, by simple division, we obtain that

$$\frac{p_X(x_2)}{p_X(x_1)} = \frac{\mathrm{e}^{-\frac{1}{2}\langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle}}{\mathrm{e}^{-\frac{1}{2}\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle}} = \mathrm{e}^{\frac{1}{2}(\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle - \langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle)}.$$

According to the **Cauchy-Schwarz inequality**, we have $|\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle| \leq ||\Sigma^{-1}(x_1-\mu)||_2 \cdot ||x_1-\mu||_2$, where $||.||_2 : \mathbb{R}^n \to [0,\infty)$ denotes the Euclidean norm. We also use the same notation $||.||_2 : \mathcal{M}_n(\mathbb{R}) \to [0,\infty)$ for the induced matrix norm, i.e., $||A||_2 = \sup_{y \in \mathbb{R}^n \setminus \{\mathbf{0}\}} \frac{||Ay||_2}{||y||_2}, \forall A \in \mathcal{M}_n(\mathbb{R})$. Clearly, it follows from the previous definition that $||\Sigma^{-1}(x_1-\mu)||_2 \leq ||\Sigma^{-1}||_2 \cdot ||x_1-\mu||_2 \Rightarrow ||\Sigma^{-1}(x_1-\mu)||_2 \cdot ||x_1-\mu||_2 \leq ||\Sigma^{-1}||_2 \cdot ||x_1-\mu||_2^2$. From this inequality and the previous Cauchy-Schwarz inequality, we obtain by transitivity that $|\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle| \leq ||\Sigma^{-1}||_2 \cdot ||x_1-\mu||_2^2 \Rightarrow \langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle \leq ||\Sigma^{-1}||_2 \cdot ||x_1-\mu||_2^2$. Similarly, it can be shown that $|\langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle| \leq ||\Sigma^{-1}||_2 \cdot ||x_2-\mu||_2^2 \Rightarrow -\langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle \leq ||\Sigma^{-1}||_2 \cdot ||x_2-\mu||_2^2$. Adding the two inequalities, we get

$$\begin{aligned}\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle - \langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle \\ \leq ||\Sigma^{-1}||_2 \cdot (||x_1-\mu||_2^2 + ||x_2-\mu||_2^2)\end{aligned}$$

.

Furthermore, from the triangle inequality, we have $||x_1-\mu||_2 \leq ||x_1-a||_2 + ||a-\mu||_2 = r_1 + ||a-\mu||_2 \Rightarrow ||x_1-\mu||_2^2 \leq (r_1 + ||a-\mu||_2)^2$, and similarly: $||x_2-\mu||_2^2 \leq (r_2 + ||a-\mu||_2)^2$. Adding these, we get $||x_1-\mu||_2^2 + ||x_2-\mu||_2^2 \leq (r_1 + ||a-\mu||_2)^2 + (r_2 + ||a-\mu||_2)^2$. Substituting again, we have

$$\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle - \langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle \leq ||\Sigma^{-1}||_2 \cdot ((r_1 + ||a-\mu||_2)^2 + (r_2 + ||a-\mu||_2)^2).$$

Thus, we obtain that

$$\begin{aligned}\frac{p_X(x_2)}{p_X(x_1)} &= \mathrm{e}^{\frac{1}{2}(\langle \Sigma^{-1}(x_1-\mu), x_1-\mu\rangle - \langle \Sigma^{-1}(x_2-\mu), x_2-\mu\rangle)} \\ &\leq \mathrm{e}^{\frac{1}{2}||\Sigma^{-1}||_2 \cdot ((r_1+||a-\mu||_2)^2 + (r_2+||a-\mu||_2)^2)}.\end{aligned}$$

1.3 The Curse of Dimensionality and Solutions

In conclusion, for a Gaussian distribution (but not exclusively), $\frac{p_X(x_2)}{p_X(x_1)}$ can be bounded by a constant. Therefore, $\frac{P(X \in \overline{B(a,r_2)})}{P(X \in \overline{B(a,r_1)})} = \frac{p_X(x_2)}{p_X(x_1)} \cdot \left(\frac{r_2}{r_1}\right)^n$ decreases exponentially to 0 as n increases. Let's discuss the relevance of this observation. We fix a **training instance**, whose value we denote as $a \in \mathbb{R}^n$, and let $r_1 \in (0,\infty)$ be the distance from a to the furthest other training instance. Under these conditions, all training data will be within $B(a, r_1)$. Let $r_2 \in (0, r_1)$ be

arbitrarily fixed. Then, $P(X \in \overline{B(a,r_2)}|X \in \overline{B(a,r_1)}) = \frac{P(X \in \overline{B(a,r_2)})}{P(X \in \overline{B(a,r_1)})} = \frac{p_X(x_2)}{p_X(x_1)} \cdot \left(\frac{r_2}{r_1}\right)^n$ becomes arbitrarily small when n is sufficiently large. So, the probability that a random observation of the vector X lies within a smaller ball $\overline{B(a,r_2)}$, *given that it is within the larger ball* $\overline{B(a,r_1)}$, tends to 0 as n tends to infinity. In other words, for high dimensions, most training instances will be at similar distances. In this context, any attempt to extrapolate certain properties of the training data as being relevant for properties of the vector X can be forms of **overfitting**, meaning we confuse entirely random properties of the data with general characteristics. This phenomenon is one of the formalizations of the concept of **the curse of dimensionality**.

Next, we will discuss **the curse of dimensionality** in the case where X is a discrete random vector. To avoid overfitting, it would be ideal to have training data that covers a number of cases comparable to the total number of possible values for X. However, since each component can take at least 2 values (otherwise, it would be redundant), the total number of cases is greater than 2^n. In other words, to avoid overfitting, the number of training instances should grow exponentially with the dimension n of the space; this is another formalization of the concept of **the curse of dimensionality**.

This formalization can be extended to any other type of random vector $X : \Omega \to \mathbb{R}^n$. We construct the vector $Y : \Omega \to \{0,1\}^n$ according to the following rule: for each $i \in \overline{1,n}$, $Y_i = 0$ if X_i is less than the empirical mean of the component X_i, and $Y_i = 1$ otherwise. Thus, if we were to use vector Y instead of vector X without a number of training instances that grows exponentially with n, we would be susceptible to overfitting for the reason explained in the previous paragraph. Under these conditions, if we are susceptible to overfitting using vector Y, which is more simplified than vector X, we will be even more susceptible if we use vector X. In conclusion, a formalization of the concept of **the curse of dimensionality** that is valid under any conditions is as follows: to avoid overfitting, the number of training instances must grow exponentially with the dimension of the space, which is very difficult in practice. Other mathematical formalizations of the curse of dimensionality can be found at the refererences [1] and [2].

One solution to mitigate **the curse of dimensionality** is **data dimensionality reduction**. This is divided into two categories: **feature selection** and **feature extraction**. Feature selection involves eliminating those components of the random vector X (called features or attributes) that do not help us identify certain patterns. Feature extraction consists of finding a function $f : \text{Im}(X) \to \mathbb{R}^d$ (where d is significantly smaller than n), such that $f(\text{Im}(X))$ approximately preserves certain patterns of $\text{Im}(X)$. In this dissertation, we will discuss feature extraction, specifically the UMAP (**Uniform Manifold Approximation and Projection**) algorithm, the algorithm that inspired it, tSNE (**t-distributed Stochastic Neighbor Embedding**), as well as the algorithm that inspired tSNE, namely SNE (**Stochastic Neighbor Embedding**).

Chapter 2

SNE Algorithm

2.1 Introduction to SNE

In this chapter, we will present the SNE (Stochastic Neighbor Embedding) algorithm as it was originally conceived [3].
Let $x_1, \ldots, x_k \in \mathbb{R}^n$ be our training data. We define a set of random vectors $\forall i \in \{1, \ldots, k\}, N_i \sim \mathcal{N}(x_i, \Sigma_i)$ which represents the selection of a "neighbor" for instance x_i. Therefore, the probability density of the vector N_i will be $p_{N_i}(x) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma_i}} \mathrm{e}^{-\frac{1}{2}\langle \Sigma_i^{-1}(x-x_i), x-x_i\rangle}$. We assume that the components of the random variable are uncorrelated (thus, they have zero covariance), and their dispersion is the same, non-zero of course, which we denote σ_i^2. Under these conditions, the covariance matrix of the vector N_i is $\Sigma_i = \sigma_i^2 I_n \Rightarrow \det \Sigma_i = \sigma_i^{2n}, \Sigma^{-1} = \frac{1}{\sigma_i^2} I_n$. Substituting into the probability density formula, we obtain $p_{N_i}(x) = \frac{1}{\sqrt{(2\pi)^n \sigma_i^{2n}}} \mathrm{e}^{-\frac{1}{2}\langle \frac{1}{\sigma_i^2}(x-x_i), x-x_i\rangle} = \frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} \mathrm{e}^{-\frac{||x-x_i||^2}{2\sigma_i^2}}$, where we use (and will continue to use throughout this dissertation) the notation $||.||$ for the Euclidean norm (only in the introductory chapter is there an ambiguity regarding the norm we are working with; in the rest of the paper, we will work only with the Euclidean norm).
Next, we would like to find a formula for $P(N_i = x_j | N_i \in \{x_s | s \in \{1, \ldots, k\}, s \neq i\})$ for any $i, j \in \{1, \ldots, k\}, i \neq j$. Applying the classical definition of conditional probability, namely that $\forall A, B \in \mathcal{F}, P(A|B) = \frac{P(A \cap B)}{P(B)}$, we would obtain an indeterminate case, since $P(N_i \in \{x_s | s \in \{1, \ldots, k\}, s \neq i\}) = 0$, as N_i is continuous and $\{x_s | s \in \{1, \ldots, k\}, s \neq i\}$ is finite. Under these conditions, we want to generalize the definition of the concept of $P(N_i = x_j | N_i \in \{x_s | s \in \{1, \ldots, k\}, s \neq i\})$. For this, we check if $\lim\limits_{\epsilon \searrow 0} P(N_i \in \overline{B(x_j, \epsilon)} | N_i \in \bigcup\limits_{s \in \{1,\ldots,k\}}^{s \neq i} \overline{B(x_s, \epsilon)})$ exists.
Let $\epsilon \in (0, \min\limits_{s,t \in \{1,\ldots,k\}} \frac{||x_s - x_t||}{2})$. Thus, an immediate consequence of the triangle inequality is that $\forall s, t \in \{1, \ldots, k\}, \overline{B(x_s, \epsilon)} \cap \overline{B(x_t, \epsilon)} = \emptyset$. From this, we obtain that $P(N_i \in \bigcup\limits_{s \in \{1,\ldots,k\}}^{s \neq i} \overline{B(x_s, \epsilon)}\}) = \sum\limits_{s \in \{1,\ldots,k\}}^{s \neq i} P(N_i \in \overline{B(x_s, \epsilon)})$. Since $j \neq i$, we have $\overline{B(x_j, \epsilon)} \subset \bigcup\limits_{s \in \{1,\ldots,k\}}^{s \neq i} \overline{B(x_s, \epsilon)} \Rightarrow \{N_i \in$

$\overline{B(x_j,\epsilon)}\} \subset \{N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}\} \Rightarrow P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)})$
$= \frac{P(N_i\in\overline{B(x_j,\epsilon)})}{P(N_i\in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)})}$. From the last two relations, we obtain by substitution that
$P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}) = \frac{P(N_i\in\overline{B(x_j,\epsilon)})}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} P(N_i\in\overline{B(x_s,\epsilon)})}$.
Furthermore, since for any $s \in \{1,\dots,k\}$, the ball $\overline{B(x_s,\epsilon)}$ is a compact and connected set, and the probability density p_{N_i} is continuous, it follows from the mean value theorem for integrals that $\exists \xi_s \in \overline{B(x_s,\epsilon)}$, such that $P(N_i \in \overline{B(x_s,\epsilon)}) = p_{N_i}(\xi_s)\lambda(\overline{B(x_s,\epsilon)})$. We showed in the introductory chapter that $\forall a_1,a_2 \in \mathbb{R}^n, r_1,r_2 \in (0,\infty), \frac{\lambda(\overline{B(a_2,r_2)})}{\lambda(\overline{B(a_1,r_1)})} = (\frac{r_2}{r_1})^n$, as well as the fact that the Lebesgue measure of any ball is strictly positive. An immediate conclusion would be that two balls with the same radius have the same strictly positive Lebesgue measure. But all balls in our problem have radius ϵ, which implies that $\exists V \in (0,\infty)$, such that $\forall s \in \{1,\dots,k\}, \lambda(\overline{B(x_s,\epsilon)}) = V$. Thus, we have $P(N_i \in \overline{B(x_s,\epsilon)}) = p_{N_i}(\xi_s)V$.
From the conclusions of the last two paragraphs, we have that
$P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}) = \frac{p_{N_i}(\xi_j)V}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i}(p_{N_i}(\xi_s)V)} = \frac{p_{N_i}(\xi_j)V}{(\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{N_i}(\xi_s))V} = \frac{p_{N_i}(\xi_j)}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{N_i}(\xi_s)}$.
To summarize, we obtained that $\forall \epsilon \in (0, \min\limits_{s,t\in\{1,\dots,k\}} \frac{||x_s-x_t||}{2}), \forall s \in \{1,\dots,k\}, \exists \xi_i \in \overline{B(x_s,\epsilon)}$, such that $P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}) = \frac{p_{N_i}(\xi_j)}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{N_i}(\xi_s)}$. Thus, by taking the limit, we obtain that $\lim\limits_{\epsilon\searrow 0} P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}) = \frac{p_{N_i}(x_j)}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{N_i}(x_s)}$.
So, we arrived at the following generalization of conditional probability: $P(N_i = x_j|N_i \in \{x_s|s \in \{1,\dots,k\}, s\neq i\}) = \lim\limits_{\epsilon\searrow 0} P(N_i \in \overline{B(x_j,\epsilon)}|N_i \in \bigcup\limits_{s\in\{1,\dots,k\}}^{s\neq i} \overline{B(x_s,\epsilon)}) = \frac{p_{N_i}(x_j)}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{N_i}(x_s)}$. We found earlier the formula $p_{N_i}(x) = \frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} e^{-\frac{||x-x_i||^2}{2\sigma_i^2}}$. Thus, substituting, we obtain: $P(N_i = x_j|N_i \in \{x_s|s \in \{1,\dots,k\}, s\neq i\}) = \frac{\frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} \frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = \frac{\frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\frac{1}{\sqrt{(2\pi\sigma_i^2)^n}} \sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = \frac{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}$.
Furthermore, we observe that $\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} P(N_i = x_j|N_i \in \{x_s|s \in \{1,\dots,k\}, s\neq i\})$

$= \sum_{j\in\{1,\ldots,k\}}^{j\neq i} \frac{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = \frac{\sum_{j\in\{1,\ldots,k\}}^{j\neq i} e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = 1$, so this extension of the conditional probability concept indeed behaves like a conditional probability. Thus, by denoting $p_{ij} = P(N_i = x_j | N_i \in \{x_s | s \in \{1,\ldots,k\}, s \neq i\}) = \frac{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}$, we obtain a discrete probability distribution.

2.2 Computing the p_{ij} probabilities

Next, the authors of the paper suggest the following rule for setting the value of σ_i: using binary search, we find its value for which the entropy $H(\{p_{ij}|j \in \{1,\ldots,n\}, j \neq i\}) = \sum_{j\in\{1,\ldots,k\}}^{j\neq i} p_{ij} \log_2 \frac{1}{p_{ij}}$ is equal to $\log_2 k_0$, where k_0 is fixed and takes values strictly less than $k-1$ and strictly greater than 1 (thus, we ensure that all distributions $\{p_{ij}|j \in \{1,\ldots,k\}, j \neq i\}$ have the same entropy regardless of i, so they are not very "different" from each other).
Thus, we want to check if we can indeed find a value for σ_i such that $H(\{p_{ij}|j \in \{1,\ldots,n\}, j \neq i\}) = \log_2 k_0$ (or at least, find sufficiently general conditions under which we can). For this, we rewrite $p_{ij} = \frac{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = \frac{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{e^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}} \sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}}}$

$= \frac{1}{\sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}}}$

$$\lim_{\sigma_i\to 0} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} = \begin{cases} 0, & \text{if } ||x_j-x_i||^2 - ||x_s-x_i||^2 < 0 \\ 1, & \text{if } ||x_j-x_i||^2 - ||x_s-x_i||^2 = 0 \\ \infty, & \text{if } ||x_j-x_i||^2 - ||x_s-x_i||^2 > 0 \end{cases}$$

Evidently, $\lim_{\sigma_i\to 0} \sum_{s\in\{1,\ldots,k\}}^{s\neq i} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} = \sum_{s\in\{1,\ldots,k\}}^{s\neq i} \lim_{\sigma_i\to 0} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}}$, from which we deduce:

$$\lim_{\sigma_i\to 0} \sum_{\substack{s\in\{1,\ldots,k\} \\ s\neq i}} e^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}}$$

$$= \begin{cases} \infty, \text{if } \exists \in \{1,\ldots,k\},\ s \neq i, \text{ such that } ||x_j-x_i||^2 - ||x_s-x_i||^2 > 0 \\ |\{s \in \{1,\ldots,k\} \mid s \neq i,\ ||x_j-x_i||^2 - ||x_s-x_i||^2 = 0\}|, \text{ otherwise} \end{cases}$$

Rewriting, we obtain:

$$\lim_{\sigma_i \to 0} p_{ij}(\sigma_i) = \begin{cases} 0, \text{ if } ||x_j - x_i|| \neq \min\limits_{s \in \{1,\dots,k\},\, s \neq i} \|x_s - x_i\|, \\ \dfrac{1}{\left|\{r \in \{1,\dots,k\} \mid r = \operatorname{argmin}\limits_{s \in \{1,\dots,k\},\, s \neq i} \|x_s - x_i\|^2\}\right|}, \text{ otherwise} \end{cases}$$

Denoting $a = |\{r \in \{1,\dots,k\} | r = \operatorname{argmin}\limits_{s \in \{1,\dots,k\}, s \neq i} ||x_s - x_i||\}|$, we obtain that $\lim\limits_{\sigma_i \to 0} H(\sigma_i) = \lim\limits_{\sigma_i \to 0} \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} p_{ij}(\sigma_i) \log_2 \frac{1}{p_{ij}(\sigma_i)} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \lim\limits_{\sigma_i \to 0} p_{ij}(\sigma_i) \log_2 \frac{1}{p_{ij}(\sigma_i)} = a \cdot \frac{1}{a} \log_2 \frac{1}{\frac{1}{a}} + (k-1-a) \cdot \lim\limits_{p \searrow 0} p \log_2 \frac{1}{p} = \log_2 a + (k-1-a) \lim\limits_{q \to \infty} \frac{\log_2 q}{q} = \log_2 a + (k-1-a) \cdot 0 = \log_2 a.$

Now, we observe that $\lim\limits_{\sigma_i \to \infty} \mathrm{e}^{-\frac{||x_s - x_i||^2}{2\sigma_i^2}} = \mathrm{e}^0 = 1, \forall s \in \{1,\dots,k\}, s \neq i.$ Substituting, we get $\lim\limits_{\sigma_i \to \infty} p_{ij}(\sigma_i) = \lim\limits_{\sigma_i \to \infty} \frac{\mathrm{e}^{-\frac{||x_j - x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{-\frac{||x_s - x_i||^2}{2\sigma_i^2}}} = \frac{1}{1 \cdot (k-1)} = \frac{1}{k-1}$. Now, we calculate $\lim\limits_{\sigma_i \to \infty} H(\sigma_i) = \lim\limits_{\sigma_i \to \infty} \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} p_{ij}(\sigma_i) \log_2 \frac{1}{p_{ij}(\sigma_i)} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \lim\limits_{\sigma_i \to \infty} p_{ij}(\sigma_i) \log_2 \frac{1}{p_{ij}(\sigma_i)} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{1}{k-1} \log_2 \frac{1}{\frac{1}{k-1}} = (k-1) \cdot \frac{1}{k-1} \log_2 (k-1) = \log_2 (k-1).$

Next, we are interested in the monotonicity of entropy H as a function of σ_i. We observe that $H = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} p_{ij} \log_2 \frac{1}{p_{ij}} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} p_{ij} \frac{\ln \frac{1}{p_{ij}}}{\ln 2} = \frac{1}{\ln 2} \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} p_{ij} \ln \frac{1}{p_{ij}}$. This rewriting not only demonstrates that the choice of logarithm base in the definition of entropy does not influence its behavior in any way, being merely a rescaling by a constant (which will justify using another base later in the algorithm), but also helps us calculate $\frac{\partial H}{\partial \sigma_i}$, which indicates the desired monotonicity.

For this, we will use the chain rule: $\frac{\partial H}{\partial \sigma_i} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial H}{\partial p_{ij}} \cdot \frac{\partial p_{ij}}{\partial \sigma_i}$. First, we calculate $\frac{\partial H}{\partial p_{ij}}$ as follows:

$\frac{\partial H}{\partial p_{ij}} = \frac{\partial}{\partial p_{ij}} (\frac{1}{\ln 2} \sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} p_{is} \ln \frac{1}{p_{is}}) = \frac{1}{\ln 2} \sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} \frac{\partial}{\partial p_{ij}} (p_{is} \ln \frac{1}{p_{is}}) = \frac{1}{\ln 2} (1 \cdot \ln \frac{1}{p_{ij}} + p_{ij} \cdot \frac{1}{\frac{1}{p_{ij}}} \cdot (-\frac{1}{p_{ij}^2})) = \frac{1}{\ln 2} (\ln \frac{1}{p_{ij}} - 1)$

Now, we also calculate $\frac{\partial p_{ij}}{\partial \sigma_i}$:

$$\frac{\partial p_{ij}}{\partial \sigma_i} = \frac{\partial}{\partial \sigma_i} \left(\frac{1}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}}} \right) = -\frac{1}{\left(\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}} \right)^2}$$

$$\cdot \frac{\partial}{\partial \sigma_i} \left(\sum_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}} \right) = -\left(\frac{1}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}}} \right)^2$$

$$\cdot \sum_{s \in \{1,\dots,k\}}^{s \neq i} \frac{\partial}{\partial \sigma_i} \left(\mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}} \right) = -p_{ij}^2 \sum_{s \in \{1,\dots,k\}}^{s \neq i} \mathrm{e}^{\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2}} \cdot \frac{\partial}{\partial \sigma_i} \left(\frac{||x_j - x_i||^2 - ||x_s - x_i||^2}{2\sigma_i^2} \right) =$$

$$
\begin{aligned}
&-p_{ij}^2 \sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} \cdot \frac{||x_j-x_i||^2-||x_s-x_i||^2}{2} \cdot \left(-\frac{2}{\sigma_i^3}\right) \\
&= \frac{p_{ij}^2}{\sigma_i^3} \sum_{s\in\{1,\dots,k\}}^{s\neq i} (||x_j-x_i||^2 - ||x_s-x_i||^2)\mathrm{e}^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} \\
&= \frac{p_{ij}^2}{\sigma_i^3} \left(||x_j-x_i||^2 \sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} - \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{\frac{||x_j-x_i||^2-||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}^2}{\sigma_i^3} \left(||x_j-x_i||^2 \frac{\sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}{\mathrm{e}^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}} - \mathrm{e}^{\frac{||x_j-x_i||^2}{2\sigma_i^2}} \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}^2}{\sigma_i^3} \left(||x_j-x_i||^2 \frac{1}{p_{ij}} - \mathrm{e}^{\frac{||x_j-x_i||^2}{2\sigma_i^2}} \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}}{\sigma_i^3} \left(||x_j-x_i||^2 - p_{ij}\mathrm{e}^{\frac{||x_j-x_i||^2}{2\sigma_i^2}} \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}}{\sigma_i^3} \left(||x_j-x_i||^2 - \frac{\mathrm{e}^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} \mathrm{e}^{\frac{||x_j-x_i||^2}{2\sigma_i^2}} \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}}{\sigma_i^3} \left(||x_j-x_i||^2 - \frac{1}{\sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} \right) \\
&= \frac{p_{ij}}{\sigma_i^3} \left(||x_j-x_i||^2 - \sum_{s\in\{1,\dots,k\}}^{s\neq i} ||x_s-x_i||^2 \frac{\mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}{\sum_{r\in\{1,\dots,k\}}^{r\neq i} \mathrm{e}^{-\frac{||x_r-x_i||^2}{2\sigma_i^2}}} \right) \\
&= \frac{p_{ij}}{\sigma_i^3} \left(||x_j-x_i||^2 - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is}||x_s-x_i||^2 \right)
\end{aligned}
$$

Next, we substitute $Z = \sum_{r\in\{1,\dots,k\}}^{r\neq i} \mathrm{e}^{-\frac{||x_r-x_i||^2}{2\sigma_i^2}} \Rightarrow p_{is} = \frac{\mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}{Z} \Rightarrow \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} = Zp_{is} \Rightarrow \mathrm{e}^{\frac{||x_s-x_i||^2}{2\sigma_i^2}} = \frac{1}{Zp_{is}} \Rightarrow ||x_s-x_i||^2 = 2\sigma_i^2 \ln \frac{1}{Zp_{is}}$. Thus, we substitute into the previous calculation and obtain:

$$
\begin{aligned}
&\frac{\partial p_{ij}}{\partial \sigma_i} = \frac{p_{ij}}{\sigma_i^3} \left(2\sigma_i^2 \ln \frac{1}{Zp_{ij}} - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is} \cdot 2\sigma_i^2 \ln \frac{1}{Zp_{is}} \right) = \frac{2p_{ij}}{\sigma_i} \left(\ln \frac{1}{Zp_{ij}} - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is} \ln \frac{1}{Zp_{is}} \right) \\
&= \frac{2p_{ij}}{\sigma_i} \left((\ln \tfrac{1}{Z} - \ln p_{ij}) - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is}(\ln \tfrac{1}{Z} - \ln p_{is}) \right) \\
&= \frac{2p_{ij}}{\sigma_i} \left(\left(1 - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is} \right) \ln \tfrac{1}{Z} - \left(\ln p_{ij} - \sum_{s\in\{1,\dots,k\}}^{s\neq i} p_{is} \ln p_{is} \right) \right)
\end{aligned}
$$

$= \frac{2p_{ij}}{\sigma_i}\left((1-1)\ln\frac{1}{Z} - \left(\ln p_{ij} - \sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{is}(-\ln\frac{1}{p_{is}})\right)\right)$

$= \frac{2p_{ij}}{\sigma_i}\left(0\cdot\ln\frac{1}{Z} - \left(\ln p_{ij} + \sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} p_{is}\ln\frac{1}{p_{is}}\right)\right)$

$= \frac{2p_{ij}}{\sigma_i}(0-(\ln p_{ij} + H\ln 2)) = \frac{2p_{ij}}{\sigma_i}(0-(H\ln 2 - \ln\frac{1}{p_{ij}})) = \frac{2p_{ij}}{\sigma_i}(\ln\frac{1}{p_{ij}} - H\ln 2)$

Next, we substitute into $\frac{\partial H}{\partial\sigma_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial H}{\partial p_{ij}}\cdot\frac{\partial p_{ij}}{\partial\sigma_i}$, and obtain:

$\frac{\partial H}{\partial\sigma_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{1}{\ln 2}(\ln\frac{1}{p_{ij}}-1)\cdot\frac{2p_{ij}}{\sigma_i}(\ln\frac{1}{p_{ij}}-H\ln 2) = \frac{2}{\sigma_i\ln 2}\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}(\ln\frac{1}{p_{ij}}-1)(\ln\frac{1}{p_{ij}}-H\ln 2) =$

$\frac{2}{\sigma_i\ln 2}\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}(\ln^2\frac{1}{p_{ij}} - (H\ln 2+1)\ln\frac{1}{p_{ij}} + H\ln 2) = \frac{2}{\sigma_i\ln 2}\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(p_{ij}\ln^2\frac{1}{p_{ij}} - p_{ij}(H\ln 2 +$

$1)\ln\frac{1}{p_{ij}} + p_{ij}H\ln 2) = \frac{2}{\sigma_i\ln 2}\left(\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\ln^2\frac{1}{p_{ij}} - (H\ln 2+1)\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\ln\frac{1}{p_{ij}} + H\ln 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\right) =$

$\frac{2}{\sigma_i\ln 2}\left(\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\ln^2\frac{1}{p_{ij}} - (H\ln 2+1)H\ln 2 + H\ln 2\cdot 1\right) = \frac{2}{\sigma_i\ln 2}\left(\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\ln^2\frac{1}{p_{ij}} - (H\ln 2)^2\right) =$

$\frac{2}{\sigma_i\ln 2}\cdot\ln^2 2\left(\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\left(\frac{\ln\frac{1}{p_{ij}}}{\ln 2}\right)^2 - \frac{(H\ln 2)^2}{\ln^2 2}\right) = \frac{2\ln 2}{\sigma_i}\left(\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\log_2^2\frac{1}{p_{ij}} - H^2\right)$

We want to simplify the expression for $\frac{\partial H}{\partial\sigma_i}$. For this, we define a discrete random variable S, as follows: $\text{Im}S = \{\log_2 p_{ij} | j\in\{1,\dots,k\}, j\neq i\}$, and $P(S=\log_2 p_{ij}) = p_{ij}, \forall j\in\{1,\dots,k\}, j\neq i$ (the probability distribution is well-defined, since $\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij} = 1$). We observe that $\mathbb{E}(S) = \sum\limits_{s\in\text{Im}(S)} P(S=s)\cdot s = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\log_2 p_{ij} = H$. Now, we calculate $\mathbb{E}(S^2) = \sum\limits_{s^2\in\text{Im}(S^2)} P(S^2 = s^2)\cdot s^2 = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\log_2^2 p_{ij}$. An immediate conclusion would be that $\text{Var}(S) = \mathbb{E}(S^2) - \mathbb{E}^2(S) = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\log_2^2 p_{ij} - H^2$.

Substituting into the previous relation, we obtain that $\frac{\partial H}{\partial\sigma_i} = \frac{2\ln 2}{\sigma_i}\text{Var}(S)$. The value $\frac{2\ln 2}{\sigma_i}$ is strictly positive, and $\text{Var}(S)$ is 0 only when S is constant, and strictly positive otherwise. Thus, we obtained that $\frac{\partial H}{\partial\sigma_i}\geq 0$, with equality only when $\text{Var}(S) = 0 \Rightarrow |\text{Im}S| = 1 \Rightarrow \log_2 p_{ij_1} = \log_2 p_{ij_2}, \forall j_1, j_2\in\{1,\dots,k\}, j_1, j_2\neq i \Rightarrow p_{ij_1} = p_{ij_2}, \forall j_1, j_2\in\{1,\dots,k\}, j_1, j_2\neq i \Rightarrow \frac{e^{-\frac{||x_{j_1}-x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} = \frac{e^{-\frac{||x_{j_2}-x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s\in\{1,\dots,k\}}^{s\neq i} e^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}}, \forall j_1, j_2\in\{1,\dots,k\}, j_1, j_2\neq i \Rightarrow e^{-\frac{||x_{j_1}-x_i||^2}{2\sigma_i^2}} = e^{-\frac{||x_{j_1}-x_i||^2}{2\sigma_i^2}}, \forall j_1, j_2\in\{1,\dots,k\}, j_1, j_2\neq i \Rightarrow ||x_{j_1}-x_i|| = ||x_{j_1}-x_i||, \forall j_1, j_2\in\{1,\dots,k\}, j_1, j_2\neq i$. Thus, we obtained that $\frac{\partial H}{\partial\sigma_i} = 0$ only when all points are equidistant from x_i. In this case, a simple calculation shows that $p_{ij} = \frac{1}{k-1}, \forall j\in\{1,\dots,k\}, j\neq i$, regardless of the choice of σ_i.

Now, let's consider the case where not all points are equidistant from x_i. In this case, we have $\frac{\partial H}{\sigma_i} > 0$, so the entropy H is strictly increasing, viewed as a function of σ_i. We previously showed that $\lim\limits_{\sigma_i\to 0} H(\sigma_i) = \log_2 a$ (where we denoted $a = |\{r\in\{1,\dots,k\} | r = \underset{s\in\{1,\dots,k\}, s\neq i}{\text{argmin}} ||x_s - x_i||\}|$),

and $\lim_{\sigma_i \to \infty} H(\sigma_i) = \log_2(k-1)$. Since entropy H, viewed as a function of σ_i, is differentiable (as shown above), it also has the Darboux property, so its image will be $(\log_2 a, \log_2(k-1))$.
Thus, we observe that the recommendation from the original paper, to fix a k_0 from 1 to $k-1$ exclusively, and then find σ_i for which $H(\sigma_i) = \log_2 k_0$, is possible only when $k_0 > a$. Indeed, $a = |\{r \in \{1, \dots, k\} | r = \underset{s \in \{1,\dots,k\}, s \neq i}{\operatorname{argmin}} ||x_s - x_i||\}|$ will most often be 1 in practice, because the chances that the distance between two points in a continuous distribution is the same are very small (theoretically, it would even be 0, but we take into account that any computer saves numbers only up to a certain decimal place, so equality in programming is, in fact, membership in a very short interval). If $a = 1$ (which usually happens), the recommended choice of σ_i is always possible. However, we will adapt the algorithm to work with any type of data.
Let us analyze what we should do when $a \geq k_0$. In this case, the infimum of the entropy (implicitly, the value closest to $\log_2 k_0$ that the entropy can attain, or at least a limit of it) would be $\log_2 a$. We have established that, if $a = k-1$, then $p_{ij} = \frac{1}{k-1}$, regardless of how we choose σ_i. Let us consider the subcase when $k_0 \leq a < k-1$. Then, we have that H is strictly increasing as a function of σ_i, and

$$\lim_{\sigma_i \to 0} H(\sigma_i) = \log_2 a,$$

so $H(\sigma_i) > \log_2 a, \forall \sigma_i \in (0, \infty)$. Therefore, H can only reach its limiting value if p_{ij} also take their limiting values. For this reason, the best option is to choose p_{ij} as $\lim_{\sigma_i \to 0} p_{ij}(\sigma_i)$. But we have computed above that:

$$\lim_{\sigma_i \to 0} p_{ij}(\sigma_i) = \begin{cases} 0, \text{ if } ||x_j - x_i|| \neq \min\limits_{s \in \{1,\dots,k\},\, s \neq i} \|x_s - x_i\|, \\ \dfrac{1}{\left|\{r \in \{1, \dots, k\} \mid r = \underset{s \in \{1,\dots,k\},\, s \neq i}{\operatorname{argmin}} \|x_s - x_i\|^2\}\right|}, \text{ otherwise} \end{cases}$$

Under these conditions, if $a \geq k_0$, we will choose: $p_{ij} = \frac{1}{a}$, if $||x_j - x_i|| = \min\limits_{s \in \{1,\dots,k\},\, s \neq i} \|x_s - x_i\|$; and $p_{ij} = 0$, otherwise.
Let us analyze the case $a < k_0$. Firstly, we re-denote $\beta_i = \frac{1}{2\sigma_i^2}$, and we have that H is strictly decreasing with respect to β_i, while $p_{ij} = \frac{1}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} e^{\beta_i(||x_j - x_i||^2 - ||x_s - x_i||^2)}}$. Next, we search for an $m > 0$ for which $H(\beta_i = m) > \log_2 k_0$, starting from an initial m which we halve until $H(\beta_i = m) > \log_2 k_0$ (we know this will happen, because $\lim_{\beta_i \to 0} H(\beta_i) = \log_2 k > \log_2 k_0$). Then, we search for an $M > 0$ for which $H(\beta_i = M) < \log_2 k_0$, starting from an initial M which we double until $H(\beta_i = M) < \log_2 k_0$ (we know this will happen, because $\lim_{\beta_i \to \infty} H(\beta_i) = \log_2 a < \log_2 k_0$). Then, we perform binary search between m and M until $|H(\beta_i) - \log_2 k_0|$ is smaller than a tolerance (finding the solution is guaranteed by applying Darboux's property on each halved interval).
Before explaining the continuation of the algorithm, the code that computes the values of $p_{ij}, \forall i, j \in \{1, \dots, k\}$ in our problem(conventionally, we will consider $p_{ii} = 0, \forall i \in \{1, \dots, k\}$) can be found in the subsection A.1.1 of the appendix.

2.3 Optimization in SNE

We will now explain the continuation of the algorithm.
Let $y_1, \dots, y_k \in \mathbb{R}^d$ (where $d < n$) be our dimensionality-reduced data. Reasoning analogously

to the calculation of probabilities p_{ij}, we calculate $q_{ij} = \frac{e^{-||y_j - y_i||^2}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} e^{-||y_s - y_i||^2}}$. The only difference is that now we have fixed all standard deviations to be equal (since we don't have a prior distribution for the data $y_1, \dots, y_k$, we don't "favor" certain points). We also chose them to be $\frac{1}{\sqrt{2}}$, so $2\sigma^2 = 1$ for simpler calculations. Our goal would be for any $i \in \{1, \dots, k\}$ to make the distributions $\{p_{ij} | j \in \{1, \dots, k\}, j \neq i\}$ and $\{q_{ij} | j \in \{1, \dots, k\}, j \neq i\}$ as similar as possible. For this, we would ideally like to minimize the Kullback-Leibler divergence between the two distributions for any $i \in \{1, \dots, k\}$: $KL(\{p_{ij} | j \in \{1, \dots, k\}, j \neq i\} || \{g_{ij} | j \in \{1, \dots, k\}) = \sum\limits_{j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln \frac{p_{ij}}{q_{ij}}$ (we have conventionally set $p_{ii} = 0$, so this is not included in the sum). Since the chances of finding a set $\{y_1, \dots, y_k\}$ that minimizes all i Kullback-Leibler divergences simultaneously are low, a compromise is to minimize their sum. Thus, the objective function will be $C(y_1, \dots, y_k) = \sum\limits_{i=1}^{k} KL(\{p_{ij} | j \in \{1, \dots, k\}, j \neq i\} || \{g_{ij} | j \in \{1, \dots, k\}) = \sum\limits_{i=1}^{k} \sum\limits_{j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln \frac{p_{ij}}{q_{ij}}$. To attempt to minimize it, we will use a variant of gradient descent with momentum, which we will describe later.

Until then, we are interested in calculating the gradient of the function C. This is equivalent to calculating $\frac{\partial C}{\partial y_i}$, where, by this notation, we mean the vector $(\frac{\partial C}{\partial y_i^1}, \dots, \frac{\partial C}{\partial y_i^d})$.

For this, we denote $\forall i, j \in \{1, \dots, k\}, r_{ij} = -||y_i - y_j||^2$. Thus, we can rewrite $q_{ij} = \frac{e^{-||y_j - y_i||^2}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} e^{-||y_s - y_i||^2}}$ $= \frac{e^{r_{ij}}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} e^{r_{is}}}$. We will use this substitution to apply the chain rule as follows: $\frac{\partial C}{\partial y_i} = \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot \frac{\partial r_{ij}}{\partial y_i}$ (since $r_{ij} = r_{ji}$, we considered it the same variable with two names, not two equal variables).

Evidently, $\frac{\partial r_{ij}}{\partial y_i} = -2(y_i - y_j)$. Substituting, we get: $\frac{\partial C}{\partial y_i} = -2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot (y_i - y_j)$.

Furthermore, we know that $C = \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \ln \frac{p_{ab}}{q_{ab}} \Rightarrow \frac{\partial C}{\partial r_{ij}} = \frac{\partial}{\partial r_{ij}} \left(\sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \ln \frac{p_{ab}}{q_{ab}} \right)$

$= \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \frac{\partial}{\partial r_{ij}} \ln \frac{p_{ab}}{q_{ab}} = \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \frac{\partial}{\partial r_{ij}} (\ln p_{ab} - \ln q_{ab}) = \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} (0 - \frac{\partial}{\partial r_{ij}} \ln q_{ab})$

$= - \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \frac{\partial}{\partial r_{ij}} \ln q_{ab} = - \left(\sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \frac{\partial}{\partial r_{ij}} \ln q_{ab} + \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{b \neq a, p_{ab} = 0} 0 \cdot \frac{\partial}{\partial r_{ij}} \ln q_{ab} \right)$

$= - \left(\sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{p_{ab} \neq 0} p_{ab} \frac{\partial}{\partial r_{ij}} \ln q_{ab} + \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{b \neq a, p_{ab} = 0} p_{ab} \frac{\partial}{\partial r_{ij}} \ln q_{ab} \right) = - \sum\limits_{a=1}^{k} \sum\limits_{b \in \{1,\dots,k\}}^{b \neq a} p_{ab} \frac{\partial}{\partial r_{ij}} \ln q_{ab}.$

Next, we try to determine $\frac{\partial}{\partial r_{ij}} \ln q_{ab}$. For this, we observe that $\ln q_{ab} = \ln \frac{e^{r_{ab}}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} e^{r_{as}}} =$

$\ln e^{r_{ab}} - \ln \sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} e^{r_{as}} = r_{ab} - \ln \sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} e^{r_{as}} \Rightarrow \frac{\partial}{\partial r_{ij}} \ln q_{ab} = \mathbf{1}_{\{i,j\}}(\{a, b\}) - \frac{1}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} e^{r_{as}}}$

$\cdot \sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} e^{r_{as}} \mathbf{1}_{\{i,j\}}(\{a, s\}) = \mathbf{1}_{\{i,j\}}(\{a, b\}) - \sum\limits_{s \in \{1,\dots,k\}}^{s \neq a} \frac{e^{r_{as}}}{\sum\limits_{t \in \{1,\dots,k\}}^{t \neq a} e^{r_{at}}} \mathbf{1}_{\{i,j\}}(\{a, s\})$

$= \mathbf{1}_{\{i,j\}}(\{a,b\}) - \sum\limits_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\mathbf{1}_{\{i,j\}}(\{a,s\})$

We substitute the previous formula into the formula for $\frac{\partial C}{r_{ij}}$ and obtain:

$$\frac{\partial C}{r_{ij}} = -\sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\left(\mathbf{1}_{\{i,j\}}(\{a,b\}) - \sum_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\mathbf{1}_{\{i,j\}}(\{a,s\})\right)$$

$$= -\left(\sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\mathbf{1}_{\{i,j\}}(\{a,b\}) - \sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\sum_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\mathbf{1}_{\{i,j\}}(\{a,s\})\right)$$

$$= -\left((p_{ij}+p_{ji}) - \sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\sum_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\left(\mathbf{1}_{(i,j)}(a,s) + \mathbf{1}_{(j,i)}(a,s)\right)\right)$$

$$= -\left((p_{ij}+p_{ji}) - \left(\sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\sum_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\mathbf{1}_{(i,j)}(a,s) + \sum_{a=1}^{k}\sum_{b\in\{1,\dots,k\}}^{b\neq a} p_{ab}\sum_{s\in\{1,\dots,k\}}^{s\neq a} q_{as}\mathbf{1}_{(j,i)}(a,s)\right)\right)$$

$$= -\left((p_{ij}+p_{ji}) - \left(\sum_{b\in\{1,\dots,k\}}^{b\neq i} p_{ib}q_{ij} + \sum_{b\in\{1,\dots,k\}}^{b\neq j} p_{jb}q_{ji}\right)\right)$$

$$= -\left((p_{ij}+p_{ji}) - \left(q_{ij}\sum_{b\in\{1,\dots,k\}}^{b\neq i} p_{ib} + q_{ji}\sum_{b\in\{1,\dots,k\}}^{b\neq j} p_{jb}\right)\right)$$

$$= -\left((p_{ij}+p_{ji}) - (q_{ij}\cdot 1 + q_{ji}\cdot 1)\right) = -(p_{ij} - q_{ij} + p_{ji} - q_{ji}).$$

Thus, we arrive at the final formula $\frac{\partial C}{\partial y_i} = -2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial C}{\partial r_{ij}}\cdot(y_i - y_j) = -2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(-(p_{ij} - q_{ij} + p_{ji} - q_{ji}))(y_i - y_j) = 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j)$

$$= 2\left(\sum_{j\in\{1,\dots,k\}}^{j\neq i}(p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j) + \sum_{j\in\{1,\dots,k\}}^{j=i} 0\cdot 0\right)$$

$$= 2\left(\sum_{j\in\{1,\dots,k\}}^{j\neq i}(p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j) + \sum_{j\in\{1,\dots,k\}}^{j=i}(p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j)\right)$$

$$= 2\sum_{j=1}^{k}(p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j).$$

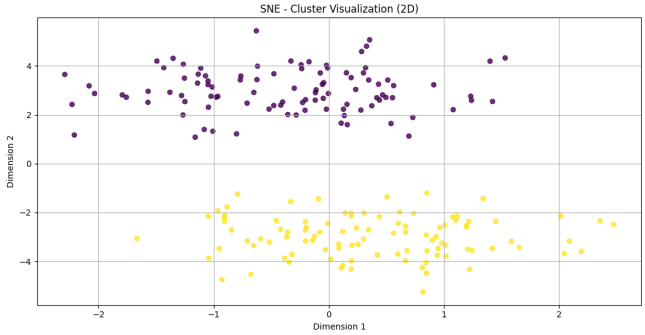
In the original reference [3], the authors describe several optimization strategies for C, without being very detailed. In this work, we will implement a variant suggested in reference [4]. They describe the following form of gradient descent with momentum:
$\Delta y_i^{t+1} = -\eta\frac{\partial C^t}{\partial y_i} + \alpha(t)\Delta y_i^t$, where $\Delta y = y_i^{t+1} - y_i^t$, and t is the epoch (iteration) we are currently in.

The learning rate η will be chosen as 0.1, and α_t will be 0.5 for the first 250 iterations, then it will increase to 0.8. Additionally, we will initialize each y_i with a randomly generated vector from a random distribution of type $\mathcal{N}(0, 10^{-4}I_d)$, as recommended in reference [5].

The continuation of the Python implementation of the SNE algorithm with the calculation of the y_i values can be found in the subsection A.1.2 from the appendix. Before this, we observe that $C = \sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln\frac{p_{ij}}{q_{ij}} = \sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}(\ln p_{ij} - \ln q_{ij}) = \sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln p_{ij} - \sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln q_{ij}$, and since $\sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln p_{ij}$ is a constant, minimizing C is equivalent to minimizing $-\sum\limits_{i=1}^{k}\sum\limits_{j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln q_{ij}$ (for calculation simplification).

Now, we will test our algorithm. To do this, we will generate 2 clusters with 100 elements each in $\mathbb{R}^{10}$. We will use the SNE algorithm to reduce the dimensionality to $\mathbb{R}^2$ and plot the corresponding points (colored differently depending on their cluster) in order to check whether the cluster membership has been preserved. The code is in the subsection A.1.3 from the appendix.

Here is the result of running this code.



As we can see, SNE preserved the clusters, therefore validating its efficiency.

Chapter 3

Symmetric SNE Algorithm

3.1 Differences between SNE and Symmetric SNE

In this chapter, we will present a slightly modified version of the SNE algorithm, namely Symmetric SNE, as described in reference [5].
The idea of this algorithm is to modify the SNE algorithm such that $p_{ij} = p_{ji}, q_{ij} = q_{ji}, \forall i, j \in \{1, \dots, k\}$, and respectively to minimize the Kullback-Leibler divergence of a single pair of probability distributions, not the sum of Kullback-Leibler divergences of multiple such pairs. Thus, we want $\sum\limits_{i,j \in \{1,\dots,k\}}^{i \neq j} p_{ij} = \sum\limits_{i,j \in \{1,\dots,k\}}^{i \neq j} q_{ij} = 1$ (by conventionally choosing $p_{ii} = q_{ii} = 0$, we can rewrite $\sum\limits_{i,j \in \{1,\dots,k\}} p_{ij} = \sum\limits_{i,j \in \{1,\dots,k\}} q_{ij} = 1$), and the new objective function to be $C = KL(\{p_{ij} | i, j \in \{1, \dots, k\}\} | \{q_{ij} | i, j \in \{1, \dots, k\}\}) = \sum\limits_{i,j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln \frac{p_{ij}}{q_{ij}} = \sum\limits_{i,j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln p_{ij} - \sum\limits_{i,j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln q_{ij}$.
In this regard, while in the original algorithm we choose $q_{ij} = \frac{e^{-||y_j - y_i||^2}}{\sum\limits_{s \in \{1,\dots,k\}}^{s \neq i} e^{-||y_s - y_i||^2}}$, we will now choose $q_{ij} = \frac{e^{-||y_j - y_i||^2}}{\sum\limits_{s,t \in \{1,\dots,k\}}^{s \neq t} e^{-||y_s - y_t||^2}}$. This formula is symmetric in i and j, so $q_{ij} = q_{ji}$. Also, we observe that $\sum\limits_{i,j \in \{1,\dots,k\}}^{i \neq j} q_{ij} = \sum\limits_{i,j \in \{1,\dots,k\}}^{i \neq j} \frac{e^{-||y_j - y_i||^2}}{\sum\limits_{s,t \in \{1,\dots,k\}}^{s \neq t} e^{-||y_s - y_t||^2}} = \frac{\sum\limits_{i,j \in \{1,\dots,k\}}^{i \neq j} e^{-||y_j - y_i||^2}}{\sum\limits_{s,t \in \{1,\dots,k\}}^{s \neq t} e^{-||y_s - y_t||^2}} = 1$, so $\{q_{ij} | i, j \in \{1, \dots, k\}, i \neq j\}$ forms a discrete probability distribution. If we maintain the convention $q_{ii} = 0, \forall i \in \{1, \dots, k\}$, then $\{q_{ij} | i, j \in \{1, \dots, k\}\}$ also forms a probability distribution. Thus, the new formula for q_{ij} satisfies the requirements discussed in the first paragraph.
Now, let's discuss how we will choose the probabilities p_{ij}. An immediate idea would be to apply the same strategy as for q_{ij} and choose the formula $p_{ij} = \frac{e^{-\frac{||x_j - x_i||^2}{2\sigma_i^2}}}{\sum\limits_{s,t \in \{1,\dots,k\}}^{s \neq t} e^{-\frac{||x_s - x_t||^2}{2\sigma_i^2}}}$. How-

ever, this approach suffers from a problem that we will describe next. First, we observe that $\sum_{s,t\in\{1,\dots,k\}}^{s\neq t} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}} = \sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}} + \sum_{s\in\{1,\dots,k\}}^{s\neq i} \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}} > \sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}} \Rightarrow \frac{1}{\sum_{s,t\in\{1,\dots,k\}}^{s\neq t} \mathrm{e}^{-\frac{||x_s-x_i||^2}{2\sigma_i^2}}} < \frac{1}{\sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}}}$. Also, we observe that if we denote $D_i = \min_{s\in\{1,\dots,k\},s\neq i} ||x_s - x_i||$, we get $\mathrm{e}^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}} \leq \mathrm{e}^{-\frac{D_i^2}{2\sigma_i^2}}$. Multiplying the two inequalities (all terms being positive), we get: $p_{ij} = \frac{\mathrm{e}^{-\frac{||x_j-x_i||^2}{2\sigma_i^2}}}{\sum_{s,t\in\{1,\dots,k\}}^{s\neq t} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}}} < \frac{\mathrm{e}^{-\frac{D_i^2}{2\sigma_i^2}}}{\sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}}}$. Now let's consider that all points are fixed, except for x_i. Then, we will have that $\lim_{D_i\to\infty} \frac{\mathrm{e}^{-\frac{D_i^2}{2\sigma_i^2}}}{\sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}}} = \frac{\lim_{D_i\to\infty} \mathrm{e}^{-\frac{D_i^2}{2\sigma_i^2}}}{\sum_{s,t\in\{1,\dots,k\}}^{i\neq s\neq t\neq i} \mathrm{e}^{-\frac{||x_s-x_t||^2}{2\sigma_i^2}}} = 0$. Since $p_{ij} > 0$, by the Squeeze Theorem, we get that $\forall j \in \{1,\dots,k\}$, $\lim_{D_i\to\infty} p_{ij} = 0$. This is problematic because if x_i is an outlier (a point very far from other points), then regardless of j, p_{ij} will be very close to 0. In such a case, all terms $p_{ij}\ln q_{ij}$ in the objective function formula $C = \sum_{i,j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln p_{ij} - \sum_{i,j\in\{1,\dots,k\}}^{p_{ij}\neq 0} p_{ij}\ln q_{ij}$ will contribute very little to the objective function's value. This means that all terms q_{ij} will also contribute very little to C's value, which implicitly means that the value of y_i will influence C's value very little. Thus, our optimization algorithm might yield a result where y_i takes a value very far from the optimal one, without the objective function's value being equally far from the minimum. Therefore, if we were to adopt the same formula for p_{ij} as for q_{ij}, we would obtain an algorithm that does not preserve the distribution of outliers.

Thus, the solution is to take $p_{ij} = \frac{p_{i|j}+p_{j|i}}{2k}$, where $p_{i|j}$ is the notation for the value of p_{ij} from the original algorithm. First, we want to demonstrate that this formula satisfies the requirements from the first paragraph. The fact that $p_{ij} = p_{ji}$ is obvious, because the formula $\frac{p_{i|j}+p_{j|i}}{2k}$ is symmetric in i and j. Now, we observe that $\sum_{i,j\in\{1,\dots,k\}} p_{ij} = \sum_{i,j\in\{1,\dots,k\}} \frac{p_{i|j}+p_{j|i}}{2k} = \frac{1}{2k}\left(\sum_{i,j\in\{1,\dots,k\}} p_{i|j} + \sum_{i,j\in\{1,\dots,k\}} p_{j|i}\right) = \frac{1}{2k}\left(\sum_{i=1}^{k}\sum_{j=1}^{k} p_{i|j} + \sum_{i=1}^{k}\sum_{j=1}^{k} p_{j|i}\right) = \frac{1}{2k}\left(\sum_{i=1}^{k} 1 + \sum_{i=1}^{k} 1\right) = \frac{1}{2k}(k+k) = 1$, so $\{p_{ij}|i,j\in\{1,\dots,k\}\}$ constitutes a discrete probability distribution.

Now, we want to demonstrate that this choice of p_{ij} values does not suffer from the same problem as the previously described choice, namely that $\forall j \in \{1,\dots,k\}$, $\lim_{D_i\to\infty} p_{ij} = 0$. We assume by contradiction that this were the case. Then, we would have $\lim_{D_i\to\infty} \sum_{j=1}^{k} p_{ij} = \sum_{j=1}^{k} \lim_{D_i\to\infty} p_{ij} = \sum_{j=1}^{k} 0 = 0$. But we observe that $\sum_{j=1}^{k} p_{ij} = \sum_{j=1}^{k} \frac{p_{i|j}+p_{j|i}}{2k} = \frac{1}{2k}\left(\sum_{j=1}^{k} p_{i|j} + \sum_{j=1}^{k} p_{j|i}\right) = \frac{1}{2k}\left(1 + \sum_{j=1}^{k} p_{j|i}\right) \geq \frac{1}{2k}$.

From this remark, an immediate conclusion would be that $0 = \lim_{D_i \to \infty} \sum_{j=1}^{k} p_{ij} \geq \frac{1}{2k}$, which is clearly absurd.
Before moving on to determining the values of y_i, we will present the code that calculates the probabilities p_{ij} in the subsection A.2.1 from the appendix. Since it is very similar to the previous code, we will only comment where it differs.

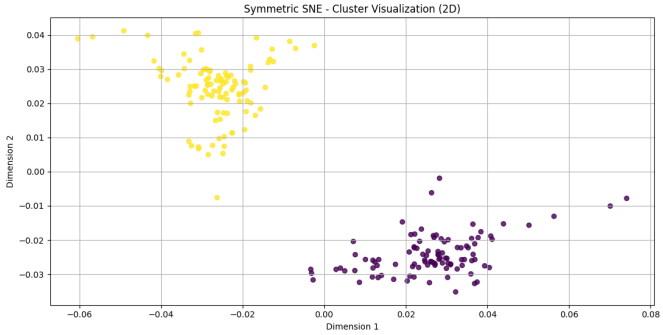
3.2 Optimization in Symmetric SNE

In what follows, we will use the same optimization method as in the previous chapter. But for this, we need to re-calculate $\frac{\partial C}{\partial y_i}$.
For this, we will apply a method similar to that in the previous chapter. We substitute $r_{ij} = -||y_j - y_i||^2$ and rewrite $q_{ij} = \frac{e^{r_{ij}}}{\sum_{s \in \{1,\dots,k\}}^{s \neq i} e^{r_{is}}}$. We apply the chain rule analogously: $\frac{\partial C}{\partial y_i} = \sum_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot \frac{\partial r_{ij}}{\partial y_i}$ (where, again, since $r_{ij} = r_{ji}$, we considered them the same variable with two different names, not two equal variables). Analogous to the previous chapter, it is evident that $\frac{\partial r_{ij}}{\partial y_i} = -2(y_i - y_j) \Rightarrow \frac{\partial C}{\partial y_i} = -2 \sum_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot (y_i - y_j)$.
Now, we need to calculate $\frac{\partial C}{\partial r_{ij}} = \frac{\partial}{\partial r_{ij}} \left(\sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \ln p_{st} - \sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \ln q_{st} \right)$
$= 0 - \frac{\partial}{\partial r_{ij}} \sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \ln q_{st} = - \sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \frac{\partial}{\partial r_{ij}} \ln q_{st}$.
Next, we observe that $\ln q_{st} = \ln \frac{e^{r_{st}}}{\sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}}} = \ln e^{r_{st}} - \ln \sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}} = r_{st} - \ln \sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}} \Rightarrow$
$\frac{\partial}{\partial r_{ij}} \ln q_{st} = \mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{\partial}{\partial r_{ij}} \ln \sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}} = \mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{1}{\sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}}} \cdot \frac{\partial}{\partial r_{ij}} \sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}} =$
$\mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{1}{\sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}}} \sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}} \mathbf{1}_{\{a,b\}=\{i,j\}} = \mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{1}{\sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}}} (e^{r_{ij}} + e^{r_{ij}}) =$
$\mathbf{1}_{\{s,t\}=\{i,j\}} - 2 \frac{e^{r_{ij}}}{\sum_{a,b \in \{1,\dots,k\}}^{a \neq b} e^{r_{ab}}} = \mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij}$.
Substituting the equality from the last paragraph into the one from the second-to-last, we get: $\frac{\partial C}{\partial r_{ij}} = - \sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st}(\mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij}) = - \left(\sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij} \sum_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \right) =$
$-((p_{ij} + p_{ji}) - 2q_{ij} \cdot 1) = -(2p_{ij} - 2q_{ij}) = -2(p_{ij} - q_{ij})$. Thus, substituting into the equality resulting from the chain rule, we have: $\frac{\partial C}{\partial y_i} = -2 \sum_{j \in \{1,\dots,k\}}^{j \neq i} (-2(p_{ij} - q_{ij})) \cdot (y_i - y_j) =$
$4 \sum_{j \in \{1,\dots,k\}}^{j \neq i} (p_{ij} - q_{ij})(y_i - y_j)$.
Now, we will display the second part of the program in the subsection A.2.2 from the appendix.

Since this is also very similar to the previous chapter, we will only comment on the changes.

Now, we will test Symmetric SNE algorithm with the same method as in the previous chapter(we generate 2 clusters in $\mathbb{R}^{10}$ and then visualise their representation reduced to $\mathbb{R}^2$ by Symmetric SNE).

Here is the result of running this code, found in the subsection A.2.3 from the appendix.



As we can see, Symmetric SNE also preserved the clusters, therefore validating its efficiency too.

Chapter 4

tSNE Algorithm

4.1 The Crowding Problem and tSNE

In this chapter, we will discuss an algorithm inspired by Symmetric SNE, namely tSNE (t-distributed Stochastic Neighbor Embedding), presented in reference [5].
Its idea stems from the observation that Symmetric SNE suffers from a problem, which the authors call "the crowding problem". Indirectly, we also touched upon it in the previous chapter when we mentioned that if p_{ij} takes a very small value, then the value of the objective function C will depend very little on the value of the probability q_{ij} (because q_{ij} only appears in the term $-p_{ij} \ln q_{ij}$ of the sum). Although we chose p_{ij}, such that $\sum_{j=1}^{k} p_{ij} \geq \frac{1}{2k}$, which does not allow all probabilities p_{ij} to become arbitrarily small, this fact does not guarantee that a particular p_{ij} cannot be very small. Under these conditions, the formula we found for p_{ij} does not completely solve the problem of the algorithm's "insensitivity" to outliers. This problem is called "the crowding problem". The name comes from the fact that if we have a very small p_{ij}, then q_{ij} should also be comparably small to have similar distributions, but the risk is precisely that in these cases, q_{ij} might be too large. Since $q_{ij} = \frac{e^{r_{ij}}}{\sum_{s,t \in \{1,\dots,k\}}^{s \neq t} e^{r_{st}}}$ (where we have kept the notation $r_{ij} = -||y_i - y_j||^2$ from previous chapters), we observe that the numerator is a constant for all probabilities $\{q_{st} | s, t \in \{1, \dots, k\}, s \neq t\}$, so its comparative value with other such probabilities strictly depends on the numerator. But the numerator is clearly increasing with respect to $r_{ij} = -||y_i - y_j||^2$, thus decreasing with respect to the distance between y_i and y_j. Therefore, q_{ij} takes larger values as y_i and y_j are closer. Under these conditions, saying that if p_{ij} is too small, there is a risk that q_{ij} is too large by comparison, is equivalent to saying that there is a risk that y_i and y_j are too close, hence the name "the crowding problem".
A solution to address the crowding problem would be to find a new distribution $\{q_{ij} | i, j \in \{1, \dots, k\}, i \neq j\}$, such that the term $\ln q_{ij}$ varies significantly with small changes in the distance between y_i and y_j (implying variations in r_{ij}) when it is already quite small (implying when r_{ij} is close to 0), so that even if p_{ij} is very small, $p_{ij} \ln q_{ij}$ still varies enough to significantly influence the value of the objective function $C = \sum_{i=1}^{k} \sum_{j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} (\ln p_{ij} - \ln q_{ij}) = \sum_{i=1}^{k} \sum_{j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln p_{ij} - \sum_{i=1}^{k} \sum_{j \in \{1,\dots,k\}}^{p_{ij} \neq 0} p_{ij} \ln q_{ij}$. This would mean having a larger value of $|\frac{\partial}{\partial r_{ij}} \ln q_{ij}|$

in a neighborhood of the point $r_{ij} = 0$. Thus, if we denote $Q_{ij} = \frac{e^{r_{ij}}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} e^{r_{st}}}$ as the probabilities from the previous chapter, and q_{ij} as those from this one, we would like to find a formula for q_{ij}, such that $|\frac{\partial}{\partial r_{ij}} \ln q_{ij}| > |\frac{\partial}{\partial r_{ij}} \ln Q_{ij}|$ in a neighborhood of the point $r_{ij} = 0$. One way to show this would be to demonstrate that $\exists \lim\limits_{r_{ij}\to 0} |\frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}}| > 1$.

The proposed solution is to use probability densities of a Cauchy distribution (which is the t-Student distribution with one degree of freedom, hence the name t-distributed Stochastic Neighbor Embedding) in the formula for q_{ij}. Thus, we will obtain the formula $q_{ij} = \frac{(1+||x_i-x_j||^2)^{-1}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1+||x_s-x_t||^2)^{-1}}$

$= \frac{(1-r_{ij})^{-1}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}}$.

Next, we will try to demonstrate that this distribution has the discussed advantage, namely that $\exists \lim\limits_{r_{ij}\to 0} |\frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}}| > 1$. We demonstrated in the previous chapter that $\frac{\partial}{\partial r_{ij}} \ln Q_{st} = \mathbf{1}_{\{s,t\}=\{i,j\}} - 2Q_{ij}, \forall s,t \in \{1,\dots,k\} \Rightarrow \frac{\partial}{\partial r_{ij}} \ln Q_{ij} = 1 - 2Q_{ij}$. We observe that $Q_{ij} = \frac{e^{r_{ij}}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} e^{r_{st}}} =$

$\frac{e^{r_{ij}}}{2e^{r_{ij}} + \sum\limits_{s,t\in\{1,\dots,k\}}^{\{s,t\}\neq\{i,j\},s\neq t} e^{r_{st}}} = \frac{1}{2 + \frac{\sum\limits_{s,t\in\{1,\dots,k\}}^{\{s,t\}\neq\{i,j\},s\neq t} e^{r_{st}}}{e^{r_{ij}}}} < \frac{1}{2} \Rightarrow 1 - 2Q_{ij} > 0 \Rightarrow |\frac{\partial}{\partial r_{ij}} \ln Q_{ij}| = \frac{\partial}{\partial r_{ij}} \ln Q_{ij} > 0.$

Now, let's also calculate $\frac{\partial}{\partial r_{ij}} \ln q_{ij} = \frac{\partial}{\partial r_{ij}} \ln \frac{(1-r_{ij})^{-1}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} =$

$\frac{\partial}{\partial r_{ij}} \left(\ln (1-r_{ij})^{-1} - \ln \sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1} \right) = \left(\frac{\partial}{\partial r_{ij}} \ln (1-r_{ij})^{-1} - \frac{\partial}{\partial r_{ij}} \ln \sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1} \right)$

$= \left(\frac{1}{(1-r_{ij})^{-1}} \cdot (1-r_{ij})^{-2} - \frac{1}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} \frac{\partial}{\partial r_{ij}} \sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1} \right)$

$= \left((1-r_{ij})^{-1} - \frac{1}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} \left((1-r_{ij})^{-2} + (1-r_{ji})^{-2} \right) \right) =$

$\left((1-r_{ij})^{-1} - \frac{1}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} \cdot 2(1-r_{ij})^{-2} \right)$

$= (1-r_{ij})^{-1} \left(1 - 2\frac{(1-r_{ij})^{-1}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} \right) = \frac{1}{1-r_{ij}}(1-2q_{ij})$. We observe that, also, $q_{ij} =$

$\frac{(1-r_{ij})^{-1}}{\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} (1-r_{st})^{-1}} = \frac{(1-r_{ij})^{-1}}{2(1-r_{ij})^{-1} + \sum\limits_{s,t\in\{1,\dots,k\}}^{\{s,t\}\neq\{i,j\},s\neq t} (1-r_{st})^{-1}} = \frac{1}{2 + \frac{\sum\limits_{s,t\in\{1,\dots,k\}}^{\{s,t\}\neq\{i,j\},s\neq t} (1-r_{st})^{-1}}{(1-r_{ij})^{-1}}} < \frac{1}{2} \Rightarrow 1 - 2q_{ij} >$

0 , and since $r_{ij} = -||y_i - y_j||^2 < 0 \Rightarrow 1 - r_{ij} > 0$, we have $\frac{\partial}{\partial r_{ij}} \ln q_{ij} = \frac{1}{1-r_{ij}}(1 - 2q_{ij}) > 0$. Since we obtained that $\frac{\partial}{\partial r_{ij}} \ln q_{ij} > 0$, as well as $\frac{\partial}{\partial r_{ij}} \ln Q_{ij} > 0$, this means that the relation $\exists \lim_{r_{ij} \to 0} |\frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}}| > 1$ that we want to demonstrate can be rewritten simply as $\exists \lim_{r_{ij} \to 0} \frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}} > 1$. Also, it means that we can divide their values and denote $\rho_{ij} = \frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}} = \frac{1}{1-r_{ij}} \cdot \frac{1-2q_{ij}}{1-2Q_{ij}}$.

Consequently, demonstrating that $\exists \lim_{r_{ij} \to 0} \frac{\frac{\partial}{\partial r_{ij}} \ln q_{ij}}{\frac{\partial}{\partial r_{ij}} \ln Q_{ij}} > 1$ is equivalent to demonstrating that $\exists \lim_{r_{ij} \to 0} \rho_{ij} > 1$.

First, we observe that $\lim_{r_{ij} \to 0} Q_{ij} = \lim_{r_{ij} \to 0} \frac{1}{\sum_{\{s,t\} \neq \{i,j\}, s \neq t} e^{r_{st}}} = \frac{1}{\sum_{\{s,t\} \neq \{i,j\}, s \neq t} e^{r_{st}}} = \frac{1}{2 + \sum_{s,t \in \{1,\dots,k\}} e^{r_{st}}}$,

respectively $\lim_{r_{ij} \to 0} q_{ij} = \lim_{r_{ij} \to 0} \frac{1}{\sum_{\{s,t\} \neq \{i,j\}, s \neq t} (1-r_{st})^{-1}} = \frac{1}{\sum_{\{s,t\} \neq \{i,j\}, s \neq t} (1-r_{st})^{-1}} = \frac{1}{2 + \sum_{s,t \in \{1,\dots,k\}} (1-r_{st})^{-1}}$.

Thus, we immediately obtain $\exists \lim_{r_{ij} \to 0} \rho_{ij} = \lim_{r_{ij} \to 0} \frac{1}{1-r_{ij}} \cdot \frac{1-2q_{ij}}{1-2Q_{ij}} = \frac{1}{1-0} \cdot \frac{1 - \frac{2}{2 + \sum_{\{s,t\} \neq \{i,j\}, s \neq t} (1-r_{st})^{-1}}}{1 - \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} e^{r_{st}}}}$

$= \frac{1 - \frac{2}{2 + \sum_{s \in \{1,\dots,k\}} (1-r_{st})^{-1}}}{1 - \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} e^{r_{st}}}}$.

From this formula, we obtain that $\lim_{r_{ij} \to 0} \rho_{ij} > 1 \Leftrightarrow \frac{1 - \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} (1-r_{st})^{-1}}}{1 - \frac{2}{2 + \sum_{\{s,t\} \neq \{i,j\}, s \neq t} e^{r_{st}}}} > 1 \Leftrightarrow 1 - \frac{2}{2 + \sum_{\{s,t\} \neq \{i,j\}, s \neq t} (1-r_{st})^{-1}} > 1 - \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} e^{r_{st}}} \Leftrightarrow \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} (1-r_{st})^{-1}} < \frac{2}{2 + \sum_{s,t \in \{1,\dots,k\}} e^{r_{st}}} \Leftrightarrow \sum_{s,t \in \{1,\dots,k\}}^{\{s,t\} \neq \{i,j\}, s \neq t} (1 - r_{st})^{-1} > \sum_{s,t \in \{1,\dots,k\}}^{\{s,t\} \neq \{i,j\}, s \neq t} e^{r_{st}}$.

Thus, it follows that if we know that $(1 - r_{st})^{-1} > e^{r_{st}}, \forall s,t \in \{1,\dots,k\}, s \neq t$, we could deduce that $\lim_{r_{ij} \to 0} \rho_{ij} > 1$. Since $r_{st} < 0, \forall s,t \in \{1,\dots,k\}, s \neq t$, this implicitly means that if we knew that $(1-x)^{-1} > e^x, \forall x \in (-\infty, 0)$, we would also deduce that $\lim_{r_{ij} \to 0} \rho_{ij} > 1$. This can be easily demonstrated as follows: $\frac{\partial}{\partial x}(e^x - (1-x)^{-1}) = e^x + (1-x)^{-2} > 0 \Rightarrow e^x - (1-x)^{-1} < e^0 - (1-0)^{-1}, \forall x \in (-\infty, 0) \Rightarrow e^x - (1-x)^{-1} < 1 - 1 = 0, \forall x \in (-\infty, 0) \Rightarrow (1-x)^{-1} > e^x$. Therefore, we have concluded that $\lim_{r_{ij} \to 0} \rho_{ij} > 1$, which was to be demonstrated.

Before continuing the description of the algorithm, we will display the corresponding Python code in the subsection A.3.1 from the appendix for what we have discussed so far. Since it is very similar to that of Symmetric SNE, we will only comment on what is different.

4.2 Optimization in tSNE

Now, we can proceed to the tSNE optimization algorithm itself. For this, we need to calculate $\frac{\partial H}{\partial y_i}$.

It is demonstrated analogously to Symmetric SNE that $\frac{\partial C}{\partial y_i} = -2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot (y_i - y_j)$, and respectively $\frac{\partial C}{\partial r_{ij}} = - \sum\limits_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \frac{\partial}{\partial r_{ij}} \ln q_{st}$.

Next, we will have $\frac{\partial}{\partial r_{ij}} \ln q_{st} = \frac{\partial}{\partial r_{ij}} \ln \frac{(1-r_{st})^{-1}}{\sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1}} = \frac{\partial}{\partial r_{ij}} \left(\ln \left((1-r_{st})^{-1} \right) - \ln \sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1} \right)$

$= \frac{\partial}{\partial r_{ij}} \ln \left((1-r_{st})^{-1} \right) - \frac{\partial}{\partial r_{ij}} \ln \sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1} = \frac{1}{(1-r_{ij})^{-1}} \cdot (1-r_{ij})^{-2} \cdot \mathbf{1}_{\{s,t\}=\{i,j\}} -$

$\frac{1}{\sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1}} \sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-2} \cdot \mathbf{1}_{\{a,b\}=\{i,j\}} = (1-r_{ij})^{-1} \cdot \mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{2(1-r_{ij})^{-2}}{\sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1}} =$

$(1-r_{ij})^{-1} \left(\mathbf{1}_{\{s,t\}=\{i,j\}} - \frac{2(1-r_{ij})^{-1}}{\sum\limits_{a,b \in \{1,\dots,k\}}^{a \neq b} (1-r_{ab})^{-1}} \right) = (1-r_{ij})^{-1} (\mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij}).$

We substitute the equality from the last paragraph into the second-to-last and obtain $\frac{\partial C}{\partial r_{ij}} =$

$- \sum\limits_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} (1-r_{ij})^{-1} (\mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij}) = -(1-r_{ij})^{-1} \sum\limits_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} (\mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij})$

$= -(1-r_{ij})^{-1} \left(\sum\limits_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \mathbf{1}_{\{s,t\}=\{i,j\}} - 2q_{ij} \sum\limits_{s,t \in \{1,\dots,k\}}^{p_{st} \neq 0} p_{st} \right) = -(1-r_{ij})^{-1} (2p_{ij} - 2q_{ij} \cdot 1)$

$= -2(p_{ij} - q_{ij})(1 + ||y_i - y_j||^2)^{-1}.$

Thus, we arrive at the conclusion $\frac{\partial C}{\partial y_i} = -2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{\partial C}{\partial r_{ij}} \cdot (y_i - y_j) = -2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} (-2(p_{ij} -$

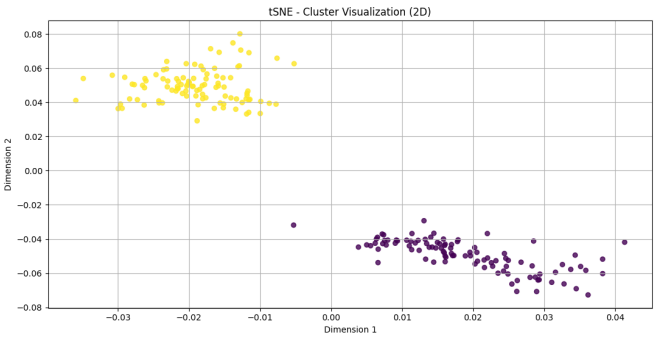
$q_{ij})(1 + ||y_i - y_j||^2)^{-1}) \cdot (y_i - y_j) = 4 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} (p_{ij} - q_{ij})(1 + ||y_i - y_j||^2)^{-1} (y_i - y_j).$

Having the new gradient formula, we can implement the algorithm A.3.2. Again, I will only comment on the Python code where it differs from Symmetric SNE.

Now, its's time to test tSNE with the same procedure as we did with SNE and Symmetric SNE.

Here is the result of running this code A.3.3.

As we can see, tSNE also preserved the clusters, therefore validating its efficiency too.


tSNE - Cluster Visualization (2D)
Dimension 2
Dimension 1
0.08
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
−0.08
−0.03
−0.02
−0.01
0.00
0.01
0.02
0.03
0.04

Chapter 5

UMAP Algorithm

5.1 UMAP and Generalized Cross-Entropy

In this chapter, we will present an algorithm inspired by tSNE, namely UMAP (Uniform Manifold Approximation and Projection). Although more topological and geometric formalizations of the algorithm can be found in reference [6], this dissertation will treat a probabilistic formalization of the algorithm.

The algorithm's idea starts from solving "the crowding problem" better than tSNE does (which only "ameliorates" it, not guaranteeing that variations of $\ln q_{ij}$ with respect to r_{ij} will be large enough to compensate for how small p_{ij} can be, but only that they are larger near $r_{ij} = 0$ than they would have been in the case of Symmetric SNE), and also from reducing the computational complexity of tSNE. To solve the crowding problem, the solution would be to use an objective function that generalizes the concept of Kullback-Leibler divergence. The authors of the article propose the following generalized cross-entropy: $C = \sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij} \ln \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \ln \frac{1-p_{ij}}{1-q_{ij}})$.

Of course, when divisions and logarithms are undefined, we will consider the limits, but there is a risk of $\frac{0}{0}$ indeterminacy. To avoid it, we rewrite: $C = \sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij}(\ln p_{ij} - \ln q_{ij}) + (1 - p_{ij})(\ln(1 - p_{ij}) - \ln(1 - q_{ij}))) = \sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij} \ln p_{ij} + (1-p_{ij}) \ln(1 - p_{ij})) - \sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij} \ln q_{ij} + (1-p_{ij}) \ln(1 - q_{ij}))$. Thus, since the term $\sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij} \ln p_{ij} + (1-p_{ij}) \ln(1 - p_{ij}))$ is a constant, minimizing C is equivalent to minimizing $C_0 = -\sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} (p_{ij} \ln q_{ij} + (1 - p_{ij}) \ln(1 - q_{ij}))$, thereby avoiding the risk of reaching the $\frac{0}{0}$ indeterminacy.

This formula no longer suffers from the crowding problem, because if p_{ij} is very small, then $1 - p_{ij}$ is very close to 1, so the term $\ln(1 - q_{ij})$, which depends on the distance between y_i and y_j, will be relevant in the objective function's value.

What we want to demonstrate is that this function retains the property of Kullback-Leibler divergence that, for two discrete probability distributions $p = (p_1, \dots, p_m)$ and $q = (q_1, \dots, q_m)$, it reaches its minimum when $p = q$.

An even stronger property than this is that for any $p \in [0, 1]$, the expression $\lim\limits_{x\to q}(-p \ln x - (1 -$

$p)\ln(1-x))$, where $p \in [0,1]$, reaches its minimum when $q = p$. If $p = 0$, then $\lim_{x \to q}(-p\ln x - (1-p)\ln(1-x)) = \lim_{x \to q}(-\ln(1-x))$, which is a strictly increasing function (composition of two strictly decreasing functions), so it reaches its minimum at the smallest feasible value, i.e., at $q = 0 = p$. If $p = 1$, then $\lim_{x \to q}(-p\ln x - (1-p)\ln(1-x)) = \lim_{x \to q}(-\ln x)$, which is a strictly decreasing function, so it reaches its minimum at the largest feasible value, i.e., at $q = 1 = p$. If $p \in (0,1)$, we define the function $f : (0,1) \to \mathbb{R}, f(q) = -p\ln q - (1-p)\ln(1-q) \Rightarrow f'(q) = -\frac{p}{q} - (-\frac{1-p}{1-q}) = \frac{1-p}{1-q} - \frac{p}{q} = \frac{(1-p)q-p(1-q)}{q(1-q)} = \frac{q-pq-p+pq}{q(1-q)} = \frac{q-p}{q(1-q)}$, which is strictly negative when q is less than p and strictly positive when q is greater than p. Therefore, f reaches its minimum at $q = p$, which will be smaller than the function's limits at the endpoints.
Under these conditions, we have obtained that the chosen objective function reaches its minimum (hypothetically, not considering the optimization problem's constraints) when the distribution q coincides with the distribution p. Not only that, but we observe that we did not use the property that $\sum p_i = \sum q_i = 1$ in this demonstration, so it is sufficient only that $p_i, q_i \in [0,1]$ for this property to hold (which is false in the case of Kullback-Leibler divergence, which would be $-\infty$, if we were to choose all q_i values to be 0, not considering that they must constitute a probability distribution).
This observation means that this generalization allows us to work to compare two sets of weights between 0 and 1 of the same cardinality, without needing to verify if these sets are probability distributions. This also solves the high computational complexity problem of tSNE, as it allows us to work with simpler formulas for p_{ij} and q_{ij}, no longer having the restriction that $\sum_{i,j \in \{1,\dots,k\}} p_{ij} = \sum_{i,j \in \{1,\dots,k\}} q_{ij} = 1$.

5.2 UMAP and Nearest Neighbor Descent

Let's first discuss how we choose the probabilities p_{ij}. Since we no longer have the restriction that these must form a probability distribution (their sum must be 1), we will no longer interpret p_{ij} as the probability that x_i chooses x_j as a neighbor among the other points, as in SNE. The simulated random experiment is that each point chooses some neighbors, but not a fixed number. However, it must happen almost surely that at least one is chosen, and almost surely that a point too far away cannot be chosen, while closer points have higher chances of being chosen than more distant ones. For this, we consider the following probability inspired by SNE/Symmetric SNE/tSNE, namely: $p_{i|j} = \mathrm{e}^{-\frac{||x_j - x_i|| - \rho_i}{\sigma_i}}$ for the k_0 closest points (where k_0 is a fixed hyperparameter), and 0 otherwise. The parameter $\sigma_i > 0$ will be chosen according to a rule, which we will describe later, and we have denoted $\rho_i = \min_{s \in \{1,\dots,k\}} ||x_s - x_i||$. Thus, we ensure that all 3 initial conditions will be met: the closest point to x_i will almost surely be chosen (because the probability is $\mathrm{e}^{-\frac{\rho - \rho}{\sigma_i}} = \mathrm{e}^0 = 1$), points too far away will almost surely not be chosen (they have probability 0), and the chances of being chosen are higher the closer they are (p_{ij} is an increasing function of $||x_j - x_i||^2$).
Next, we would like to symmetrize the probabilities. For this, we consider x_i and x_j to be neighbors if at least one chose the other as a neighbor. Thus, if we denote $A_{i|j}$ as the event "x_i chooses x_j as a neighbor", and A_{ij} as the event "x_i and x_j are neighbors", then we could write $A_{ij} = A_{i|j} \cup A_{j|i} \Rightarrow p_{ij} = P(A_{ij}) = P(A_{i|j} \cup A_{j|i}) = P(A_{i|j}) + P(A_{j|i}) - P(A_{i|j} \cap A_{j|i}) = p_{i|j} + p_{j|i} - P(A_{i|j} \cap A_{j|i})$. Furthermore, we assume that $A_{i|j}$ and $A_{j|i}$ are independent, meaning $P(A_{i|j} \cap A_{j|i}) = p_{i|j}p_{j|i}$. Substituting, we obtain $p_{ij} = p_{i|j} + p_{j|i} - p_{i|j}p_{j|i}$, thus we obtained a symmetric expression in i and j for p_{ij}.

We observe that the algorithm assumes finding the k_0 closest points for each point (in the remainder of the section, we will use the word "neighbor" in this sense, not the one from the previous paragraphs). Since finding all k_0 nearest neighbors for each point is a computationally complex problem, we will use a heuristic to solve the problem approximately. The authors of the article recommend the Nearest Neighbor Descent algorithm, which can be found in reference [7]. We will explain this heuristic further.
The principle from which Nearest Neighbor Descent starts is that neighbors of neighbors are likely to be neighbors. Thus, we randomly assign k_0 neighbors to each point in the initialization phase. Then, we apply the algorithm for a certain number of iterations. In each iteration, we apply the following procedure to each point: we take the set formed by its neighbors and the neighbors of its neighbors and select the k_0 closest points to be the new neighbors for the next iteration. Of course, if after an iteration, no point has changed its neighbors, then we no longer continue the algorithm, because we would redundantly repeat the same iteration that does not change any point's neighbors.
In the subsection A.4.1 from the appendix, you can see the implementation of a code that receives the set $\{x_1, \ldots, x_k\}$ and returns a sparse matrix, which on each row i, column j, will have $||x_j - x_i|| - \rho_i$ if x_j is one of the k_0 nearest neighbors of x_i.

5.3 UMAP and Binary Search

Next, we want to find the value of the hyperparameter σ_i. The authors of the article suggest using binary search to find the value $\sigma_i > 0$ for which: $\sum\limits_{j=1}^{k} p_{i|j} = \log_2 k_0$. For this, we want to demonstrate that such a σ_i always exists.
If we denote $\{i_1, \ldots, i_{k_0}\}$ as the set of indices of the k_0 closest neighbors of x_i, then we will have $\sum\limits_{j=1}^{k} p_{i|j} = 0 + \sum\limits_{j=1}^{k_0} p_{i|i_j} = \sum\limits_{j=1}^{k_0} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}}$. Thus, we observe that $\lim\limits_{\sigma_i \searrow 0} \sum\limits_{j=1}^{k} p_{i|j} = \lim\limits_{\sigma_i \searrow 0} \sum\limits_{j=1}^{k_0} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}}$
$= \sum\limits_{j=1}^{k_0} \lim\limits_{\sigma_i \searrow 0} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}} = \sum\limits_{j=1}^{k_0} 0 = 0$, respectively $\lim\limits_{\sigma_i \to \infty} \sum\limits_{j=1}^{k} p_{i|j} = \lim\limits_{\sigma_i \to \infty} \sum\limits_{j=1}^{k_0} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}}$
$= \sum\limits_{j=1}^{k_0} \lim\limits_{\sigma_i \to \infty} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}} = \sum\limits_{j=1}^{k_0} \mathrm{e}^0 = k_0 \cdot \mathrm{e}^0 = k_0$.
We define the function $f : (0, \infty) \to \mathbb{R}, f(x) = x - \log_2 x \Rightarrow f'(x) = 1 - \frac{1}{x \ln 2}$, which has a strictly positive derivative for any $x \geq 1$, implying it is strictly increasing on $[1, \infty)$, from which it follows that $f(k_0) > f(1) = 1 - \log_2 1 = 1 > 0 \Rightarrow k_0 > \log_2 k_0$. Also, since $k_0 > 1$, we also know that $\log_2 k_0 > 0$. Thus, we have that $\lim\limits_{\sigma_i \searrow 0} \sum\limits_{j=1}^{k} p_{i|j} = 0$, respectively $\lim\limits_{\sigma_i \to \infty} \sum\limits_{j=1}^{k} p_{i|j} = k_0$, and $\log_2 k_0 \in (0, k_0)$. Since $\sum\limits_{j=1}^{k} p_{i|j}$ has the Darboux property (being a composition of elementary operations), it means that $\exists \sigma_i \in (0, \infty)$ such that $\sum\limits_{j=1}^{k} p_{i|j}(\sigma_i) = \log_2 k_0$.
To find this, we will initialize a variable m with 1 which we will halve until $\sum\limits_{j=1}^{k} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{m}} < \log_2 k_0$ (we know this will happen, because $\lim\limits_{\sigma_i \searrow 0} \sum\limits_{j=1}^{k_0} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}} = 0 < \log_2 k_0$. Then, we will initialize a variable M with 1 which we will double until $\sum\limits_{j=1}^{k} \mathrm{e}^{-\frac{||x_{i_j} - x_i|| - \rho_i}{M}} > \log_2 k_0$ (we know

this will happen, because $\lim_{\sigma_i \to \infty} \sum_{j=1}^{k_0} e^{-\frac{||x_{i_j} - x_i|| - \rho_i}{\sigma_i}} = k_0 > \log_2 k_0$. Then, we will search for a σ_i such that $|\sum_{j=1}^{k} p_{i|j}(\sigma_i) - \log_2 k_0|$ is less or equal than a tolerance (applying the Darboux property on each bisected interval guarantees that we will find a solution).
Of course, after that, we will apply the formula $p_{ij} = p_{i|j} + p_{j|i} - p_{i|j}p_{j|i}$ to compute the p_{ij} probabilities. The code for what we have discussed in this section can be found in the subsection A.4.2 from the appendix(since it is similar to the binary search in the previous algorithms, even simpler as we don't have multiple cases, we won't comment it too much):

5.4 UMAP and Laplacian Eigenmaps

As we will see further, UMAP, like t-SNE, is also based on a gradient descent-type method, so we will first discuss how we will initialize the points $y_1, \ldots, y_k \in \mathbb{R}^d$. In previous algorithms, we started with a random initialization. However, since finding the k_0 nearest neighbors of each point $x_1, \ldots, x_n \in \mathbb{R}^n$ is a first step in another data reduction algorithm, namely Laplacian eigenmaps, we will use the output of this algorithm to initialize the values $y_1, \ldots, y_k \in \mathbb{R}^d$. More details about this algorithm can be found in reference [8].
The first step of this algorithm is to compute a matrix of weights between any two points x_i and x_j, a matrix which usually depends on the choice of the k_0 nearest neighbors of each point. Of course, in this case, we will adapt Laplacian eigenmaps to UMAP and choose the matrix that has the value p_{ij} on row i and column j. We will denote this matrix by W. We will also use the notation D for the matrix that contains $\sum_{j=1}^{k} p_{ij}$ on row i, column i, for any $i \in \{1, \ldots, k\}$; and 0 everywhere else outside the main diagonal. Another notation used will be $L = D - W$.
Next, the idea of the algorithm is to minimize (we will discuss the constraints later) the objective function $\sum_{i,j=1}^{k} p_{ij}||y_i - y_j||^2$. Thus, if x_i and x_j are close, then p_{ij} will be large, and the larger p_{ij} is, the more the objective function will "penalize" the term $||y_i - y_j||^2$ (so, if x_i and x_j are very close, then y_i and y_j will also remain very close).
Next, we will first take the simpler case where $d = 1$, then generalize to the case $d > 1$. In this case, the objective function will be $\sum_{i,j=1}^{k} p_{ij}(y_i - y_j)^2 = \sum_{i,j=1}^{k} p_{ij}(y_i^2 + y_j^2 - 2y_iy_j) = \sum_{i,j=1}^{k} p_{ij}y_i^2 + \sum_{i,j=1}^{k} p_{ij}y_j^2 - 2\sum_{i,j=1}^{k} p_{ij}y_iy_j = \sum_{i=1}^{k} y_i^2 \sum_{j=1}^{k} p_{ij} + \sum_{j=1}^{k} y_j^2 \sum_{i=1}^{k} p_{ij} - 2\sum_{i,j=1}^{k} p_{ij}y_iy_j = 2\sum_{i=1}^{k} y_i^2 \sum_{j=1}^{k} p_{ij} - 2\sum_{i,j=1}^{k} p_{ij}y_iy_j = 2\left(\sum_{i=1}^{k} y_i^2 \sum_{j=1}^{k} p_{ij} - \sum_{i,j=1}^{k} p_{ij}y_iy_j\right)$. Next, if we denote $y = (y_1, \ldots, y_k)^t$, we observe that $\sum_{i=1}^{k} y_i^2 \sum_{j=1}^{k} p_{ij} = \langle Dy, y\rangle$, and $\sum_{i,j=1}^{k} p_{ij}y_iy_j = \langle Wy, y\rangle$. Substituting, we obtain that the objective function is $2(\langle Dy, y\rangle - \langle Wy, y\rangle) = 2\langle (D-W)y, y\rangle = 2\langle Ly, y\rangle$. Since 2 is a constant, we can choose the objective function to be $\langle Ly, y\rangle$. Of course, if we do not impose any restriction, then we would have the trivial solution $y = \overline{0}$, which is obviously useless, as it would reduce all points to the same point. The recommended restriction, initially, by the authors of the algorithm is $\langle Dy, y\rangle = 1$.
Next, we observe that $\forall i \in \{1, \ldots, k\}, \exists j \in \{1, \ldots, k\}$ such that $||x_i - x_j|| = \rho_i \Rightarrow p_{i|j} =$

$e^{-\frac{||x_i-x_j||-\rho_i}{\sigma_i}} = e^{-\frac{\rho_i-\rho_i}{\sigma_i}} = e^0 = 1 \Rightarrow p_{ij} = p_{i|j} + p_{j|i} - p_{i|j}p_{j|i} = 1 + p_{j|i} - 1 \cdot p_{j|i} = 1 \Rightarrow \sum_{j=1}^{k} p_{ij} > 0$, so all elements on the main diagonal of matrix D are strictly positive(the other ones being 0), so D is symmetric and positive definite, therefore $\langle Dy, y\rangle = 0 \Leftrightarrow y = 0$. So, we have that exists $\{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\} = \{\langle L\frac{y}{\sqrt{\langle Dy,y\rangle}}, \frac{y}{\sqrt{\langle Dy,y\rangle}}\rangle | y \neq \overline{0}\}$. Since we have that $\forall y \neq \overline{0}, \langle D\frac{y}{\sqrt{\langle Dy,y\rangle}}, \frac{y}{\sqrt{\langle Dy,y\rangle}}\rangle = \frac{\langle Dy,y\rangle}{\langle Dy,y\rangle} = 1$, we deduce that $\{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\} = \{\langle L\frac{y}{\sqrt{\langle Dy,y\rangle}}, \frac{y}{\sqrt{\langle Dy,y\rangle}}\rangle | y \neq \overline{0}\} \subseteq \{\langle Ly, y\rangle | \langle Dy, y\rangle = 1\} = \{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | \langle Dy, y\rangle = 1\} \subseteq \{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\}$. We have thus obtained, by double inclusion, that $\{\langle Ly, y\rangle | \langle Dy, y\rangle = 1\} = \{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\} \Rightarrow \min\{\langle Ly, y\rangle | \langle Dy, y\rangle = 1\} = \min\{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\}$.

So, we obtained that we need to minimize the function $\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle}$ for $y \neq \overline{0}$. For this, we observe that the matrix D has elements only on the main diagonal(which are positive), so we can write $D = D^{\frac{1}{2}} \cdot D^{\frac{1}{2}}$, where $D^{\frac{1}{2}}$ denotes the matrix obtained by taking the square root of each element on the main diagonal and preserving the zero values elsewhere (evidently $(D^{\frac{1}{2}})^t = D^{\frac{1}{2}}$). Thus, we rewrite $\langle Dy, y\rangle = y^t Dy = y^t D^{\frac{1}{2}} D^{\frac{1}{2}} y = y^t (D^{\frac{1}{2}})^t D^{\frac{1}{2}} y = (D^{\frac{1}{2}} y)^t D^{\frac{1}{2}} y = \langle D^{\frac{1}{2}} y, D^{\frac{1}{2}} y\rangle = ||D^{\frac{1}{2}} y||^2$. Substituting $z = D^{\frac{1}{2}} y$, we obtain $\langle Dy, y\rangle = ||z||^2$.

Also, we proved that $\sum_{j=1}^{k} p_{ij} > 0 \Rightarrow \sqrt{\sum_{j=1}^{k} p_{ij}} > 0$, so all elements on the main diagonal of matrix $D^{\frac{1}{2}}$ are non-zero, from which we deduce that $\exists D^{-\frac{1}{2}} = (D^{\frac{1}{2}})^{-1}$ and it is the matrix that has $\frac{1}{\sqrt{\sum_{j=1}^{k} p_{ij}}}$ on row i, column i; and 0, everywhere else outside the main diagonal (implicitly, we have that $(D^{-\frac{1}{2}})^t = D^{-\frac{1}{2}}$). Thus, we obtain that $z = D^{\frac{1}{2}} y \Rightarrow y = D^{-\frac{1}{2}} z \Rightarrow \langle Ly, y\rangle = \langle LD^{-\frac{1}{2}} z, D^{-\frac{1}{2}} z\rangle = (D^{-\frac{1}{2}} z)^t LD^{-\frac{1}{2}} z = z^t (D^{-\frac{1}{2}})^t LD^{-\frac{1}{2}} z = z^t D^{-\frac{1}{2}} LD^{-\frac{1}{2}} z = \langle D^{-\frac{1}{2}} LD^{-\frac{1}{2}} z, z\rangle$. Thus, substituting $\tilde{L} = D^{-\frac{1}{2}} LD^{-\frac{1}{2}}$, we obtain $\langle Ly, y\rangle = \langle \tilde{L} z, z\rangle$.

In conclusion, we can rewrite $\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} = \frac{\langle \tilde{L}z,z\rangle}{||z||^2}$. Since we know that $z = D^{\frac{1}{2}} y$, and $D^{\frac{1}{2}}$ is invertible, we deduce that $z \neq \overline{0} \Leftrightarrow y \neq \overline{0}$, so $\min\{\langle Ly, y\rangle | \langle Dy, y\rangle = 1\} = \min\{\frac{\langle Ly,y\rangle}{\langle Dy,y\rangle} | y \neq \overline{0}\} = \min\{\frac{\langle \tilde{L}z,z\rangle}{||z||^2} | z \neq \overline{0}\}$. Since $z = D^{\frac{1}{2}} y$ is just a rescaling of y, for simplicity we can consider that $z_1, \ldots, z_k$ are the reduced values of the points $x_1, \ldots, x_n$. Thus, we can consider that z will, in fact, be the new y.

So, for now, we have the following optimization problem $\min\{\frac{\langle \tilde{L}y,y\rangle}{||y||^2} | y \neq \overline{0}\}$. To address this, we will first demonstrate some properties of the matrix $\tilde{L}$. We have symmetrized the weights p_{ij}, so we know that $\forall i, j \in \{1, \ldots, k\}, p_{ij} = p_{ji} \Rightarrow W^t = W$. Since D has elements only on the main diagonal, it is evident that $D^t = D$. From this, we deduce that $L^t = (D - W)^t = D^t - W^t = D - W = L$. From this, it follows that $\tilde{L}^t = (D^{-\frac{1}{2}} LD^{-\frac{1}{2}})^t = (D^{-\frac{1}{2}})^t L^t (D^{-\frac{1}{2}})^t = D^{-\frac{1}{2}} LD^{-\frac{1}{2}} = \tilde{L}$. So, we have found that $\tilde{L}$ is a symmetric matrix (of course, with real values), so its eigenvalues will also be real.

Next, if we were to prove that $\tilde{L}$ is also positive semi-definite, we would know that its eigenvalues are also non-negative. To do this, we rewrite $\tilde{L} = D^{-\frac{1}{2}} LD^{-\frac{1}{2}} = D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}} = D^{-\frac{1}{2}} DD^{-\frac{1}{2}} - D^{-\frac{1}{2}} WD^{-\frac{1}{2}} = I_k - D^{-\frac{1}{2}} WD^{-\frac{1}{2}}$. Next, let $z \in \mathbb{R}^k$ be arbitrarily fixed. Then, we will have $\langle \tilde{L}z, z\rangle = \langle (I_k - D^{-\frac{1}{2}} WD^{-\frac{1}{2}})z, z\rangle = \langle I_k z, z\rangle - \langle D^{-\frac{1}{2}} WD^{-\frac{1}{2}} z, z\rangle = ||z||^2 - \langle D^{-\frac{1}{2}} WD^{-\frac{1}{2}} z, z\rangle = \sum_{i=1}^{k} z_i^2 - \langle D^{-\frac{1}{2}} WD^{-\frac{1}{2}} z, z\rangle$. We denote $d_i = \sum_{j=1}^{k} p_{ij}, \forall i \in \{1, \ldots, k\}$. By a simple calculation, we obtain that the matrix $D^{-\frac{1}{2}} WD^{-\frac{1}{2}}$ will have the element $\frac{p_{ij}}{\sqrt{d_i d_j}}$ on row i, column j, for

any $i, j \in \{1, \ldots, k\}$. From this, we deduce that $\langle D^{-\frac{1}{2}} W D^{-\frac{1}{2}} z, z \rangle = \sum_{i,j=1}^{k} \frac{p_{ij}}{\sqrt{d_i d_j}} z_i z_j$. Substituting, we have: $\langle \tilde{L} z, z \rangle = \sum_{i=1}^{k} z_i^2 - \sum_{i,j=1}^{k} \frac{p_{ij}}{\sqrt{d_i d_j}} z_i z_j$. Consider the expression $\sum_{i,j=1}^{k} p_{ij} (\frac{z_i}{\sqrt{d_i}} - \frac{z_j}{\sqrt{d_j}})^2 = \sum_{i,j=1}^{k} p_{ij} (\frac{z_i^2}{d_i} + \frac{z_j^2}{d_j} - 2\frac{z_i z_j}{\sqrt{d_i d_j}}) = \sum_{i,j=1}^{k} p_{ij} \frac{z_i^2}{d_i} + \sum_{i,j=1}^{k} p_{ij} \frac{z_j^2}{d_j} - 2 \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = \sum_{i=1}^{k} \frac{z_i^2}{d_i} \sum_{j=1}^{k} p_{ij} + \sum_{j=1}^{k} \frac{z_j^2}{d_j} \sum_{i=1}^{k} p_{ij} - 2 \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = \sum_{i=1}^{k} \frac{z_i^2}{d_i} d_i + \sum_{j=1}^{k} \frac{z_j^2}{d_j} d_j - 2 \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = \sum_{i=1}^{k} z_i^2 + \sum_{j=1}^{k} z_j^2 - 2 \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = 2 \sum_{i=1}^{k} z_i^2 - 2 \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = 2(\sum_{i=1}^{k} z_i^2 - \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}}) \Rightarrow \sum_{i=1}^{k} z_i^2 - \sum_{i,j=1}^{k} p_{ij} \frac{z_i z_j}{\sqrt{d_i d_j}} = \frac{1}{2} \sum_{i,j=1}^{k} p_{ij} (\frac{z_i}{\sqrt{d_i}} - \frac{z_j}{\sqrt{d_j}})^2$. Applying transitivity to the equalities in the last two sentences, we obtain that $\langle \tilde{L} z, z \rangle = \frac{1}{2} \sum_{i,j=1}^{k} p_{ij} (\frac{z_i}{\sqrt{d_i}} - \frac{z_j}{\sqrt{d_j}})^2 \geq 0, \forall z \in \mathbb{R}^k$, so the matrix $\tilde{L}$ is positive semi-definite.

Thus, we have concluded that all eigenvalues of the matrix $\tilde{L}$ are non-negative. We enumerate the eigenvalues of the matrix as follows: $0 \leq \lambda_0 \leq \lambda_1 \leq \ldots \leq \lambda_{k-1}$. Also, the fact that $\tilde{L}$ is symmetric means that there exists an orthonormal basis of eigenvectors of the matrix $\tilde{L}$ in $\mathbb{R}^k$, which we denote by $\{f_0, \ldots, f_{k-1}\}$ (each vector corresponding to the eigenvalue with the same index, i.e., $\forall i \in \{1, \ldots, k\}, \tilde{L} f_i = \lambda_i f_i$). Thus, we know that $\exists \alpha_0, \ldots, \alpha_{k-1} \in \mathbb{R}$ such that $y = \sum_{i=0}^{k-1} \alpha_i f_i \Rightarrow \frac{\langle \tilde{L} y, y \rangle}{||y||^2} = \frac{\left\langle \tilde{L} \sum_{i=0}^{k-1} \alpha_i f_i, \sum_{i=0}^{k-1} \alpha_i f_i \right\rangle}{|| \sum_{i=0}^{k-1} \alpha_i f_i ||^2} = \frac{\left\langle \sum_{i=0}^{k-1} \alpha_i \tilde{L} f_i, \sum_{i=0}^{k-1} \alpha_i f_i \right\rangle}{|| \sum_{i=0}^{k-1} \alpha_i f_i ||^2} = \frac{\left\langle \sum_{i=0}^{k-1} \alpha_i \lambda_i f_i, \sum_{i=0}^{k-1} \alpha_i f_i \right\rangle}{|| \sum_{i=0}^{k-1} \alpha_i f_i ||^2}$.

Since the basis $\{f_0, \ldots, f_{k-1}\}$ is orthonormal, we have $\left\langle \sum_{i=0}^{k-1} \alpha_i \lambda_i f_i, \sum_{i=0}^{k-1} \alpha_i f_i \right\rangle = \sum_{i=0}^{k-1} \alpha_i^2 \lambda_i$, respectively $|| \sum_{i=0}^{k-1} \alpha_i f_i ||^2 = \sum_{i=0}^{k-1} \alpha_i^2$. Substituting, we obtain: $\frac{\langle \tilde{L} y, y \rangle}{||y||^2} = \frac{\sum_{i=0}^{k-1} \alpha_i^2 \lambda_i}{\sum_{i=0}^{k-1} \alpha_i^2} = \sum_{i=0}^{k-1} \frac{\alpha_i^2}{\sum_{j=0}^{k-1} \alpha_j^2} \lambda_i$. It is evident that the minimum value of this convex combination is λ_0, which is attained only if $\alpha_i = 0, \forall i \in \{1, \ldots, k-1\}, \lambda_i \neq \lambda_0$, but $(\alpha_0, \ldots, \alpha_{k-1}) \neq \overline{0}$. This would mean that y is a linear combination of the eigenvectors of the basis $\{f_0, \ldots, f_{k-1}\}$, corresponding to the smallest eigenvalue, which we denote $\{f_0, \ldots, f_{i_0}\}$. We know that the eigenvectors corresponding to the same eigenvalue form a linear subspace, whose dimension is the geometric multiplicity of that eigenvalue. In the case of diagonalizable matrices (such as $\tilde{L}$, since it is symmetric), the geometric multiplicity equals the algebraic multiplicity (i.e. i_0). Thus, since $f_0, \ldots, f_{i_0}$ are orthogonal to each other, they are also linearly independent, so they form a basis for the subspace of eigenvectors corresponding to the eigenvalue λ_0 (because their number is fixed i_0, which is the dimension of the subspace). To recap, we obtained that the objective function reaches its minimum if and only if y is a linear combination of the elements of a basis of the linear subspace of eigenvectors corresponding to the eigenvalue λ_0, which is equivalent to belonging to the subspace. Reformulating, the final conclusion is that the minimum is achieved when y is an arbitrary eigenvector corresponding to the smallest eigenvalue.

Next, we will observe that the approach presented above suffers from a problem and we will discuss how to adjust it. For this, we want to check if $\lambda_0 = 0$. Then, we want to solve the equation $\tilde{L} f_0 = 0 \Rightarrow D^{-\frac{1}{2}} L D^{-\frac{1}{2}} f_0 = 0 \Rightarrow L D^{-\frac{1}{2}} f_0 = D^{\frac{1}{2}} \cdot 0 = 0$. We make the substitution $v = D^{-\frac{1}{2}} f_0$ and rewrite $Lv = 0 \Rightarrow \langle Lv, v \rangle = 0$. At the beginning of the section, we started with the objective

function $\sum_{i,j=1}^{k} p_{ij}(y_i - y_j)^2$, which we demonstrated could be rewritten in matrix form as $2\langle Ly, y\rangle$. Thus, we have $0 = \langle Lv, v\rangle = \frac{\sum_{i,j=1}^{k} p_{ij}(v_i - v_j)^2}{2} \Rightarrow \sum_{i,j=1}^{k} p_{ij}(v_i - v_j)^2 = 0$. From this, we obtain that this happens whenever $v_i = v_j, \forall i, j \in \{1, \ldots, k\}, p_{ij} \neq 0$. The problem is that this is a trivial dimensionality reduction, because without the rescaling consisting of multiplication by $D^{\frac{1}{2}}$ (since $v = D^{-\frac{1}{2}} f_0 \Rightarrow f_0 = D^{\frac{1}{2}} v$), we would have too many points reduced to the same point.

Under these conditions, we will add an additional constraint to our optimization problem, namely that y must belong to the vector subspace orthogonal to the vector subspace generated by the eigenvectors corresponding to 0 (this orthogonal subspace is non-empty, because there is at least one eigenvalue different from 0, since the matrix $\tilde{L}$ is non-zero, as there is a non-zero p_{ij}, which means $L = D - W$ will have the value $-p_{ij}$ on row i, column j, which implies $\tilde{L} = D^{-\frac{1}{2}} L D^{-\frac{1}{2}}$ will have the value $-\frac{p_{ij}}{\sqrt{d_i d_j}}$ on row i, column j). Let λ_{i_0+1} be the first eigenvalue different from 0. Then, we know that $\exists \alpha_{i_0+1}, \ldots, \alpha_k \in \mathbb{R}$ such that $y = \sum_{i=i_0+1}^{k} \alpha_i f_i$. Then, by an analogous reasoning to the paragraph before the previous one, it is shown that the minimum is λ_{i_0+1}, which is attained when y is some eigenvector corresponding to the eigenvalue λ_{i_0+1}.

Now, let's discuss how we will proceed in the general case when $d > 1$. The objective function we started with was $\sum_{i,j=1}^{k} p_{ij}||y_i - y_j||^2 = \sum_{i,j=1}^{k} p_{ij} \sum_{s=1}^{d} (y_i^s - y_j^s)^2 = \sum_{s=1}^{d} \sum_{i,j=1}^{k} p_{ij}(y_i^s - y_j^s)^2$. For any $s \in \{1, \ldots, d\}$, we denote $y^s = (y_1^s, \ldots, y_k^s)^t$. It is shown analogously to the $d = 1$ case that $\sum_{i,j=1}^{k} p_{ij}(y_i^s - y_j^s)^2 = 2\langle Ly^s, y^s\rangle$. Substituting, we obtain $\sum_{i,j=1}^{k} p_{ij}||y_i - y_j||^2 = \sum_{s=1}^{d} 2\langle Ly^s, y^s\rangle = 2\sum_{s=1}^{d} \langle Ly^s, y^s\rangle$. Subjecting each term of the sum to the same restrictions and rescalings as in the $d = 1$ case, we arrive at the new objective function $\sum_{s=1}^{d} \frac{\langle \tilde{L}y^s, y^s\rangle}{||y^s||^2}$, of course with the condition $\forall s \in \{1, \ldots, k\}, y^s \neq \overline{0}$. If we were to impose the same restriction as in the case $d = 1$ on each term in the sum, then the solution to the optimization problem would be that each y^s is an eigenvector of the same eigenvalue, which would represent a trivial dimensionality reduction. To avoid this situation, we will sequentially minimize each term of the sum, inductively imposing the additional restriction that each y^s must be orthogonal to the linear subspace generated by the eigenvectors corresponding to eigenvalues smaller than λ_{i_0+s}. By an analogous reasoning to the $d = 1$ case, we obtain that y^s must be some eigenvector corresponding to the eigenvalue λ_{i_0+s}.

A problem with this approach is that it only works if d is less than or equal to $k - m_0$, where $m_0 = i_0 + 1$ denotes the multiplicity of 0 (because by inductively applying the procedure described above, we would end up imposing the condition that y^s is orthogonal to all eigenvectors, thus to the entire space, if $s > k - m_0$). This problem rarely occurs in practice, because as we discussed in the introductory chapter of this dissertation, to avoid the curse of dimensionality, the number of training data must be exponentially larger than the number of attributes. So, if we encounter the problem described above when applying Laplacian eigenmaps, most likely we have chosen too large a value for d. Nevertheless, we will implement a solution for this case as well: we will apply Laplacian eigenmaps to reduce the problem to dimension $k - m_0$, then we will iteratively concatenate the solution matrix until we reach dimension d (of course, if d is not divisible by $k - m_0$, then at the end we will only add the first r eigenvectors corresponding to a

non-zero eigenvalue, where r is the remainder of d divided by $k - m_0$).
Now, let's return to the details of implementing the base case when $d \leq k - m_0$. We want to find eigenvectors from positions $m_0 + 1, \ldots, m_0 + d$, sorted by their eigenvalues in ascending order. For this, we would like to calculate a computationally efficient way to find the value of m_0 (and which is stable to machine precision rounding). For this, we consider the weighted graph with adjacency matrix $\tilde{L}$. We aim to show that the number of connected components of the graph equals the multiplicity of the eigenvalue 0. Since $\tilde{L}$ is symmetric, it is diagonalizable, so its algebraic and geometric multiplicities of 0 are equal. Note that the rank of a matrix is invariant under simultaneous row and column permutations. By reordering the vectors $x_1, \ldots, x_k$, the matrices D, $D^{-1/2}$, and W undergo identical permutations of their rows and columns. Consequently, the matrix $\tilde{L} = D^{-1/2}(D - W)D^{-1/2}$ is conjugated by a permutation matrix, and thus its rank remains unchanged. From the rank-nullity theorem, we know that k is the sum of the dimension of the kernel and the dimension of the image of the linear operator associated with $\tilde{L}$(which is its rank). Therefore, the permuted $\tilde{L}$ will also have the same kernel dimension, and thus the same geometric multiplicity of 0. Hence, the geometric multiplicity of 0 is preserved. Since the matrix $\tilde{L}$ remains symmetric after reordering the vectors $x_1, \ldots, x_k$(for the same reasons as the former $\tilde{L}$ was symmetric), the algebraic multiplicity of 0 is the same as its geometric multiplicity. For this reason, we will refer to the multiplicity of 0 without distinguishing between the geometric and algebraic notions.
We have obtained that we can permute $x_1, \ldots, x_k$ as we wish, and the multiplicity of 0 remains the same, so we choose the following rule: first, the points from the first connected component, then those from the second, ..., finally, those from the last connected component. Let m be the number of connected components. Then, the matrix W will have the form $\operatorname{diag}(W_1, \ldots, W_m)$, where W_i is the probability matrix associated with connected component $i \in \{1, \ldots, m\}$. Evidently, using the same conventions, we will have $D = \operatorname{diag}(D_1, \ldots, D_m)$, implicitly $\tilde{L} = D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}} = \operatorname{diag}(D_1^{-\frac{1}{2}}(D_1 - W_1)D_1^{-\frac{1}{2}}, \ldots, D_m^{-\frac{1}{2}}(D_m - W_m)D_m^{-\frac{1}{2}}) = \operatorname{diag}(\tilde{L_1}, \ldots, \tilde{L_m})$. Let's assume there are at least 2 connected components with indices $i, j \in \{1, \ldots, m\}, i \neq j$. Then, if we select a row of $\tilde{L}$ corresponding to a row of $\tilde{L_i}$, and a row of $\tilde{L}$ corresponding to a row of $\tilde{L_j}$, then the two rows will be linearly independent, because each will take the value 0 on columns where the other is non-zero. Under these conditions, the maximum number of linearly independent rows of the matrix $\tilde{L}$ will be the sum of the maximum number of linearly independent rows of each matrix $\tilde{L_i}$. Reformulating, the rank of $\tilde{L}$ is the sum of the ranks of the matrices $\tilde{L_i}$, i.e., $\operatorname{rank} \tilde{L} = \sum_{i=1}^{m} \operatorname{rank} \tilde{L_i}$. If we denote by m_i the multiplicity of 0 in the case of matrix $\tilde{L_i}$ (which we discussed is the dimension of the kernel) and k_i the cardinality of connected component $i \in \{1, \ldots, m\}$, then we apply the rank-nullity theorem to the equality in the previous sentence and obtain: $k - m_0 = \sum_{i=1}^{m}(k_i - m_i) = \sum_{i=1}^{m} k_i - \sum_{i=1}^{m} m_i = k - \sum_{i=1}^{m} m_i \Rightarrow m_0 = \sum_{i=1}^{m} m_i$.
Thus, if we demonstrate that $\forall i \in \{1, \ldots, k\}, m_i = 1$, we could immediately deduce that $m_0 = m$, which is what needs to be proven. In other words, our problem reduces to showing that if our graph is connected, then $\tilde{L}$ has eigenvalue 0, and its multiplicity is 1. For this, we want to solve the equation $\tilde{L}f = 0$. We have already approached this equation in this section and demonstrated that if we substitute $v = D^{-\frac{1}{2}}f$, then we conclude that $v_i = v_j, \forall i, j \in \{1, \ldots, k\}, p_{ij} \neq 0$. But our hypothesis is that the graph is connected, i.e., $\forall i, j \in \{1, \ldots, k\}, i \neq j, \exists (i_0 = i, i_1, \ldots, i_t = j)$ such that $p_{ii_1}, \ldots, p_{i_{t-1}j} \neq 0 \Rightarrow v_i = v_{i_1} = \ldots = v_j$. Therefore, it follows that $v_i = v_j, \forall i, j \in \{1, \ldots, k\}, i \neq j$, i.e., $\exists \alpha \in \mathbb{R}$ such that $v = \alpha \cdot \overline{1} \Rightarrow f = D^{\frac{1}{2}}v = \alpha D^{\frac{1}{2}}\overline{1}$. Next, we verify the found solution: $\tilde{L}f = D^{-\frac{1}{2}}LD^{-\frac{1}{2}} \cdot \alpha D^{\frac{1}{2}}\overline{1} = \alpha D^{-\frac{1}{2}}LD^{-\frac{1}{2}}D^{\frac{1}{2}}\overline{1} = \alpha D^{-\frac{1}{2}}L\overline{1} = \alpha D^{-\frac{1}{2}}(D - W)\overline{1} = \alpha D^{-\frac{1}{2}}(D\overline{1} - W\overline{1})$. We observe that both $D\overline{1}$ and $W\overline{1}$ will have $\sum_{j=1}^{k} p_{ij}$ on row i, so

$D\overline{1} = W\overline{1} \Rightarrow D\overline{1} - W\overline{1} = 0 \Rightarrow \alpha D^{-\frac{1}{2}}(D\overline{1} - W\overline{1}) = 0 \Rightarrow \tilde{L}f = 0$, which needed to be verified. So, we obtained that 0 is an eigenvalue, and the eigenvectors generated by it are those generated by $D^{\frac{1}{2}}\overline{1}$. Therefore, the multiplicity of the eigenvalue 0 is 1 for each connected component, which was to be demonstrated.
We will attach a Python implementation of a Laplacian eigenmap in the subsection A.4.3 from the appendix.

5.5 UMAP and Non-Linear Least Squares

Now that we have established how we will initialize the points $y_1, \ldots, y_k \in \mathbb{R}^d$, let's discuss how we will choose the formula for q_{ij}. As mentioned in the first section of this chapter, the generalized cross-entropy formula no longer comes with the restriction that $\sum_{i,j=1}^{k} q_{ij} = 1$ like the Kullback-Leibler divergence, so we will choose a formula inspired by tSNE, but much simpler, namely $q_{ij} = (1 + a||y_i - y_j||^{2b})^{-1}$, where $a, b > 0$ are 2 hyperparameters. We would like there to be a form of similarity between this choice for the values of q_{ij} and the choice of the values $p_{i|j} = \mathrm{e}^{-\frac{||x_j - x_i|| - \rho_i}{\sigma_i}}$ for the closest k_0 points (where ρ_i is the distance to the nearest point), and 0 otherwise.
Thus, we define the family of functions $\Phi_{a,b} : \{x \in [0, \infty) | 1 + ax^{2b} \neq 0\} \to \mathbb{R}, \Phi_{a,b}(x) = (1 + ax^{2b})^{-1}$, and the function

$$\Psi(x) = \begin{cases} 1, & \text{if } x \leq d_{min}, \\ \mathrm{e}^{-(x - d_{min})}, & \text{otherwise.} \end{cases}$$

, where $d_{min} > 0$ is a fixed hyperparameter, and $(a, b) \neq (-1, 0)$ (otherwise, the function $\Phi_{a,b}$ would not be defined, we would have a division by 0). For the choice of values for a and b, we will use Non-Linear Least Squares to minimize $\int_0^\infty (\Phi_{a,b}(x) - \Psi(x))^2 dx$. For this, we want to study the conditions under which the integral is convergent and demonstrate that $a > 0, b > 0$, for any local minimum (a, b) of the objective function.
It is a known and evident property that if $\lim_{x \to \infty} (\Phi_{a,b}(x) - \Psi(x))^2 > 0$, then $\int_0^\infty (\Phi_{a,b}(x) - \Psi(x))^2 dx$ diverges to $+\infty$. To eliminate some cases from the discussion, we are interested in analyzing this limit in various cases. We observe that $\lim_{x \to \infty} \Psi(x) = \lim_{x \to \infty} \mathrm{e}^{-(x - d_{min})} = \mathrm{e}^{-\infty} = 0$. If $b < 0$, we have that $\lim_{x \to \infty} x^{2b} = 0 \Rightarrow \lim_{x \to \infty} \Phi_{a,b}(x) = \frac{1}{1 + a \cdot 0} = 1 \Rightarrow \lim_{x \to \infty} (\Phi_{a,b}(x) - \Psi(x))^2 = (1 - 0)^2 = 1 > 0$, so the objective function diverges to $+\infty$ in this case. If $b = 0$, we have that $\Phi_{a,0}(x) = \frac{1}{1 + ax^0} = \frac{1}{1 + a} \Rightarrow \lim_{x \to \infty} (\Phi_{a,b}(x) - \Psi(x))^2 = (\frac{1}{1 + a} - 0)^2 = \frac{1}{(1 + a)^2} > 0$, so the objective function diverges to $+\infty$, similarly. In the case when $a = 0$, we have that $\Phi_{0,b}(x) = \frac{1}{1 + 0 \cdot x^b} = 1 \Rightarrow \lim_{x \to \infty} (\Phi_{a,b}(x) - \Psi(x))^2 = (1 - 0)^2 = 1 > 0$, so the objective function diverges to $+\infty$, again.
Through the procedure in the previous paragraph, we have deduced that the only situations where the integral could possibly converge are when $b > 0$ and $a \neq 0$. Let's first analyze the case when $a < 0$ and $b > 0$. Then, we observe that the function $\Phi_{a,b}$ has a point where it is not defined on the interval $(0, \infty)$, namely $x_0 = \sqrt[2b]{-\frac{1}{a}}$. Under these conditions, we have that the objective function is the sum of two improper integrals, specifically $\int_0^\infty (\Phi_{a,b}(x) - \Psi(x))^2 dx = \int_0^{x_0} (\Phi_{a,b}(x) - \Psi(x))^2 dx + \int_{x_0}^\infty (\Phi_{a,b}(x) - \Psi(x))^2 dx$. Let's first analyze the first of these, $\int_0^{x_0} (\Phi_{a,b}(x) - \Psi(x))^2 dx$. We observe that $a < 0, b > 0, x_0 = \sqrt[2b]{-\frac{1}{a}} \Rightarrow \forall x \in$

$(0, x_0), 0 < 1 + ax^{2b} < 1 \Rightarrow \forall x \in (0, x_0), \Phi_{a,b}(x) = (1 + ax^{2b})^{-1} > 1$. We also observe that $\forall x > d_{min}, \mathrm{e}^{-(x-d_{min})} < 1 \Rightarrow \forall x \in [0, \infty), \Psi(x) \leq 1 \Rightarrow \forall x \in (0, x_0), \Phi_{a,b}(x) > 1 \geq \Psi(x) \Rightarrow \forall x \in (0, x_0), \Phi_{a,b}(x) - \Psi(x) \geq \Phi_{a,b}(x) - 1 > 0 \Rightarrow \forall x \in (0, x_0), (\Phi_{a,b}(x) - \Psi(x))^2 \geq (\Phi_{a,b}(x) - 1)^2 \Rightarrow \int_0^{x_0} (\Phi_{a,b}(x) - \Psi(x))^2 dx \geq \int_0^{x_0} (\Phi_{a,b}(x) - 1)^2 dx = \int_0^{x_0} (\frac{1}{1+ax^{2b}} - 1)^2 dx = \int_0^{x_0} (\frac{ax^{2b}}{1+ax^{2b}})^2 dx$. We make the substitution $t = -ax^{2b} \Rightarrow x = \sqrt[2b]{-\frac{1}{a}} \cdot t^{\frac{1}{2b}} \Rightarrow dx = \sqrt[2b]{-\frac{1}{a}} \cdot \frac{1}{2b} t^{\frac{1}{2b}-1} dt$. We observe that $t(0) = -a \cdot 0^{2b} = 0$, and $ax_0^{2b} + 1 = 0 \Rightarrow -t(x_0) + 1 = 0 \Rightarrow t(x_0) = 1$. We substitute into the previous integral: $\int_0^{x_0} (\frac{ax^{2b}}{1+ax^{2b}})^2 dx = \int_0^1 (\frac{-t}{1-t})^2 \sqrt[2b]{-\frac{1}{a}} \cdot \frac{1}{2b} t^{\frac{1}{2b}-1} dt = \frac{1}{2b} \sqrt[2b]{-\frac{1}{a}} \int_0^1 \frac{t^2}{(1-t)^2} t^{\frac{1}{2b}-1} dt = \frac{1}{2b} \sqrt[2b]{-\frac{1}{a}} \int_0^1 t^{1+\frac{1}{2b}} (1-t)^{-2} dt$. It is a known fact that the Beta function $B : \mathbb{R}^2 \to [0, \infty], B(u, v) = \int_0^1 t^{u-1}(1-t)^{v-1} dt$ is finite if and only if $u, v > 0$. In our case, we have that $\int_0^{x_0} (\frac{ax^{2b}}{1+ax^{2b}})^2 dx = \frac{1}{2b} \sqrt[2b]{-\frac{1}{a}} \int_0^1 t^{1+\frac{1}{2b}} (1-t)^{-2} dt = \frac{1}{2b} \sqrt[2b]{-\frac{1}{a}} B(2 + \frac{1}{2b}, -1) = \frac{1}{2b} \sqrt[2b]{-\frac{1}{a}} \cdot \infty = \infty$, because $b > 0$. Also, we have that $(\Phi_{a,b}(x) - \Psi(x))^2 \geq 0 \Rightarrow \int_{x_0}^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \geq 0 \Rightarrow \int_0^{x_0} (\Phi_{a,b}(x) - \Psi(x))^2 dx + \int_{x_0}^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \geq \int_0^{x_0} (\Phi_{a,b}(x) - \Psi(x))^2 dx = \infty \Rightarrow \int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx = \infty$, so we have obtained that the objective function diverges to $+\infty$ in this case as well.

Therefore, the only case left for us to check would be the one where $a > 0$ and $b > 0$. Under these conditions, we observe that both $(\Phi_{a,b}(x) - \Psi(x))^2$, and x^{-4b} are well-defined on $[1, \infty)$, and $\lim_{x\to\infty} \frac{(\Phi_{a,b}(x)-\Psi(x))^2}{x^{-4b}} = \lim_{x\to\infty} \frac{((1+ax^{2b})^{-1}-0)^2}{x^{-4b}} = \lim_{x\to\infty} \frac{((x^{2b}(x^{-2b}+a))^{-1})^2}{x^{-4b}} = \lim_{x\to\infty} \frac{x^{-4b}(x^{-2b}+a)^{-2}}{x^{-4b}} = \lim_{x\to\infty} (x^{-2b}+a)^{-2} = a^{-2} > 0$. Thus, we deduce through the limit comparison test that $\int_1^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \sim \int_1^{\infty} x^{-4b} dx$. We also know that $(\Phi_{a,b}(x) - \Psi(x))^2$ is well-defined on the interval $[0, 1]$, so $\int_0^1 (\Phi_{a,b}(x) - \Psi(x))^2 dx < \infty \Rightarrow \int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx = \int_0^1 (\Phi_{a,b}(x) - \Psi(x))^2 dx + \int_1^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \sim \int_1^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \sim \int_1^{\infty} x^{-4b} dx$. Here, we first consider the subcase when $4b \leq 1 \Rightarrow \forall x \in [0, \infty), x^{-4b} \geq x^{-1} \Rightarrow \int_1^{\infty} x^{-4b} dx \geq \int_1^{\infty} x^{-1} dx = \ln x \Big|_1^{\infty} = \infty - 0 = \infty$, and since $\int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \sim \int_1^{\infty} x^{-4b} dx$, we obtain that the objective function diverges to $+\infty$ in this subcase as well. Now, let's consider the subcase when $b > \frac{1}{4} \Rightarrow \int_1^{\infty} x^{-4b} dx = \frac{x^{1-4b}}{1-4b} \Big|_1^{\infty} = 0 - \frac{1}{1-4b} = -\frac{1}{1-4b}$, so $\int_1^{\infty} x^{-4b} dx$ is convergent, and $\int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx \sim \int_1^{\infty} x^{-4b} dx$, so $\int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx$ is also convergent.

In conclusion, we have obtained that the objective function, the integral $\int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx$, is convergent only if $a > 0$ and $b > \frac{1}{4}$; and diverges to $+\infty$ in all other cases. This means that any potential local minimum will have the properties $a > 0$ and $b > \frac{1}{4}$. Thus, the formula for $q_{ij} = (1 + a||y_i - y_j||^{2b})^{-1}$ has the desired properties: it belongs to the interval $[0, 1]$ (thus, it is a probability), and it is decreasing as a function of $||y_i - y_j||$ (the farther apart two points are, the lower the probability that they are neighbors). Also, the fact that b cannot be arbitrarily close to 0 guarantees that q_{ij} varies significantly as a function of $||y_i - y_j||$.

In subsection A.4.4 in the appendix, we will display a Python implementation for Non-Linear Least Squares for the minimization of $\int_0^{\infty} (\Phi_{a,b}(x) - \Psi(x))^2 dx$.

5.6 UMAP and Stochastic Gradient Descent

Now that we have established how we will initialize the data $y_1, \ldots, y_k$, let's discuss how we will continue optimizing the objective function $C_0 = -\sum_{i,j \in \{1,\ldots,k\}}^{j \neq i} (p_{ij} \ln q_{ij} + (1 - p_{ij}) \ln (1 - q_{ij}))$. We

observe that if we denote $C_1 = -\sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i} p_{ij}\ln q_{ij}$ and $C_2 = -\sum\limits_{i,j\in\{1,\dots,k\}}^{j\neq i}(1-p_{ij})\ln(1-q_{ij})$, we have $C_0 = C_1 + C_2$.
We will firstly calculate $\frac{\partial C_1}{\partial y_i}$. We will substitute $r_{ij} = ||y_i - y_j||^2, \forall i,j \in \{1,\dots,k\}, i \neq j$ (we will consider r_{ji} to be the same variable as r_{ij}, not two equal variables), from which we have $q_{ij} = (1+ar_{ij}^b)^{-1}$. Thus, we apply the chain rule and obtain: $\frac{\partial C_1}{\partial y_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial C_1}{\partial r_{ij}}\cdot\frac{\partial r_{ij}}{\partial y_i}$. Next, we have $\frac{\partial C_1}{\partial r_{ij}} = \frac{\partial}{\partial r_{ij}}\left(-\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} p_{st}\ln q_{st}\right) = \frac{\partial}{\partial r_{ij}}\left(-\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t} p_{st}\ln(1+ar_{st}^b)^{-1}\right) = -\frac{\partial}{\partial r_{ij}}(p_{ij}\ln(1+ar_{ij}^b)^{-1}) - \frac{\partial}{\partial r_{ij}}(p_{ji}\ln(1+ar_{ji}^b)^{-1}) - 0 = -2\frac{\partial}{\partial r_{ij}}(p_{ij}\ln(1+ar_{ij}^b)^{-1})$
$= -2p_{ij}\frac{\partial}{\partial r_{ij}}\ln(1+ar_{ij}^b)^{-1} = -2p_{ij}\frac{1}{(1+ar_{ij}^b)^{-1}}\frac{\partial}{\partial r_{ij}}(1+ar_{ij}^b)^{-1} = -2p_{ij}(1+ar_{ij}^b)\cdot(-1)(1+ar_{ij}^b)^{-2}\frac{\partial}{\partial r_{ij}}(1+ar_{ij}^b) = 2p_{ij}(1+ar_{ij}^b)^{-1}abr_{ij}^{b-1} = \frac{2abr_{ij}^{b-1}}{1+ar_{ij}^b}p_{ij} = \frac{2ab||y_i-y_j||^{2(b-1)}}{1+a||y_i-y_j||^{2b}}p_{ij}$. Also, it is immediately observed that $\frac{\partial r_{ij}}{\partial y_i} = \frac{\partial}{\partial y_i}||y_i-y_j||^2 = 2(y_i-y_j)$. Substituting into the formula resulting from the chain rule, we obtain $\frac{\partial C_1}{\partial y_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial C_1}{\partial r_{ij}}\cdot\frac{\partial r_{ij}}{\partial y_i} = 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{2ab||y_i-y_j||^{2(b-1)}}{1+a||y_i-y_j||^{2b}}p_{ij}(y_i-y_j)$.
Since b can be less than 1, we may have a case of **undetermination** in the calculation of $||y_i-y_j||^{2(b-1)}$ if $||y_i-y_j|| = 0$. To avoid numerical errors, we will approximate $||y_i-y_j||^{2(b-1)} = \frac{||y_i-y_j||^{2b}}{||y_i-y_j||^2} \sim \frac{||y_i-y_j||^{2b}}{\epsilon_1+||y_i-y_j||^2}$, where $\epsilon_1 > 0$ is a hyperparameter fixed to a very small value. Under these conditions, we use the approximation $\frac{\partial C_1}{\partial y_i} \sim 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)}p_{ij}(y_i-y_j)$.

Next, we want to calculate $\frac{\partial C_2}{\partial y_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial C_2}{\partial r_{ij}}\cdot\frac{\partial r_{ij}}{\partial y_i}$. Thus, we have $\frac{\partial C_2}{\partial r_{ij}}$
$= \frac{\partial}{\partial r_{ij}}\left(-\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t}(1-p_{st})\ln(1-q_{st})\right) = \frac{\partial}{\partial r_{ij}}\left(-\sum\limits_{s,t\in\{1,\dots,k\}}^{s\neq t}(1-p_{st})\ln(1-(1+ar_{st}^b)^{-1})\right) = -\frac{\partial}{\partial r_{ij}}((1-p_{ij})\ln(1-(1+ar_{ij}^b)^{-1})) - \frac{\partial}{\partial r_{ij}}((1-p_{ji})\ln(1-(1+ar_{ji}^b)^{-1})) - 0 = -2\frac{\partial}{\partial r_{ij}}((1-p_{ij})\ln(1-(1+ar_{ij}^b)^{-1})) = -2(1-p_{ij})\frac{\partial}{\partial r_{ij}}\ln(1-(1+ar_{ij}^b)^{-1}) = -2(1-p_{ij})\frac{1}{1-(1+ar_{ij}^b)^{-1}}\frac{\partial}{\partial r_{ij}}(1-(1+ar_{ij}^b)^{-1}) = -2(1-p_{ij})\frac{1}{1-(1+ar_{ij}^b)^{-1}}\cdot(-1)\frac{\partial}{\partial r_{ij}}(1+ar_{ij}^b)^{-1} = 2(1-p_{ij})\frac{1}{1-(1+ar_{ij}^b)^{-1}}\cdot(-1)(1+ar_{ij}^b)^{-2}\frac{\partial}{\partial r_{ij}}(1+ar_{ij}^b) = -2(1-p_{ij})\frac{1}{1-(1+ar_{ij}^b)^{-1}}(1+ar_{ij}^b)^{-2}abr_{ij}^{b-1} = -2(1-p_{ij})\frac{1}{((1+ar_{ij}^b)-1)(1+ar_{ij}^b)}abr_{ij}^{b-1}$
$= -2(1-p_{ij})\frac{1}{ar_{ij}^b(1+ar_{ij}^b)}abr_{ij}^{b-1} = -\frac{2b}{r_{ij}(1+ar_{ij}^b)}(1-p_{ij}) = -\frac{2b}{||y_i-y_j||^2(1+a||y_i-y_j||^{2b})}(1-p_{ij})$. Thus, we obtain $\frac{\partial C_2}{\partial y_i} = \sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{\partial C_2}{\partial r_{ij}}\cdot\frac{\partial r_{ij}}{\partial y_i} = 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(-\frac{2b}{||y_i-y_j||^2(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j))$. To avoid division by 0 if $y_i = y_j$, we will use the approximation
$\frac{\partial C_2}{\partial y_i} \sim 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(-\frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j))$, where ϵ_2 is a very small chosen hyperparameter (the authors of the article recommend $\epsilon_2 = 0.001$).
To recap, we obtained $\frac{\partial C_1}{\partial y_i} \sim 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}\frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)}p_{ij}(y_i-y_j)$, respectively
$\frac{\partial C_2}{\partial y_i} \sim 2\sum\limits_{j\in\{1,\dots,k\}}^{j\neq i}(-\frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j))$. But, in gradient descent, the

solution is moved in the opposite direction of the gradient, because the opposite direction of the gradient is a direction of decrease.

Under these conditions, we rewrite $-\frac{\partial C_1}{\partial y_i} \sim 2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} (-\frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}(y_i-y_j))$, respectively $-\frac{\partial C_2}{\partial y_i} \sim 2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j)$. Substituting $F_{ij}^a = -\frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}(y_i-y_j)$ and $F_{ij}^r = \frac{2b}{(\epsilon+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j)$, we have $-\frac{\partial C_1}{\partial y_i} = 2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} F_{ij}^a$, respectively $-\frac{\partial C_2}{\partial y_i} \sim 2 \sum\limits_{j \in \{1,\dots,k\}}^{j \neq i} F_{ij}^r$.

Let $\eta > 0$ and calculate $y_i + \eta F_{ij}^a = y_i - \eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}(y_i-y_j) = (1 - \eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}) y_i - (-\eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}) y_j \Rightarrow (y_i + \eta F_{ij}^a) - y_j = (1 - \eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}) y_i - (1 - \eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}) y_j = (1-\eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij})(y_i-y_j) \Rightarrow ||(y_i+\eta F_{ij}^a)-y_j|| = |1-\eta \frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)} p_{ij}| \cdot ||y_i-y_j||$. We observe that for a sufficiently small non-negative η, $||(y_i+\eta F_{ij}^a)-y_j|| < ||y_i-y_j||$, which means that, in a neighborhood of y_i, the direction F_{ij}^a "attracts" y_i towards y_j.

Now, let's calculate $y_i + \eta F_{ij}^r = y_i + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j) = (1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})) y_i - \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij}) y_j \Rightarrow (y_i + \eta F_{ij}^r) - y_j = (1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})) y_i - (1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})) y_j = (1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij}))(y_i-y_j) \Rightarrow ||(y_i + \eta F_{ij}^r) - y_j|| = |1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})| \cdot ||y_i-y_j|| = (1 + \eta \frac{2b}{(\epsilon_2+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})) \cdot ||y_i-y_j|| > ||y_i-y_j||$. So, we obtained that the direction F_{ji}^r "repels" y_i from y_j.

Under these conditions, the authors of the article call F_{ij}^a the **attractive force** exerted by y_j on y_i, and F_{ij}^r the **repulsive force** exerted by y_j on y_i. For easier implementation, we observe that we can rewrite $F_{ij}^a = -\frac{2ab||y_i-y_j||^{2(b-1)}}{1+a||y_i-y_j||^{2b}} p_{ij}(y_i-y_j) = -(2ab||y_i-y_j||^{2(b-1)}) q_{ij} p_{ij}(y_i-y_j)$, respectively $F_{ij}^r = \frac{2b}{(\epsilon+||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-p_{ij})(y_i-y_j) = \frac{2b}{(\epsilon+||y_i-y_j||^2)} q_{ij}(1-p_{ij})(y_i-y_j)$.

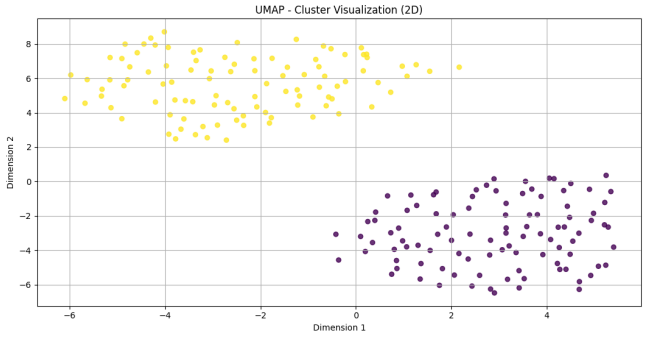
Next, we will describe the stochastic gradient descent variant proposed by the authors of the article. The idea starts from the fact that if k_0 is significantly smaller than k (and this should be the case, otherwise the computational complexity would be too large), then the majority of the values p_{ij} will be equal to 0 (this also being the reason why we use a sparse matrix to save these values). Under these conditions, the majority of attractions between points will be null, because p_{ij} appears in the factorization of F_{ij}^a, and the majority of repulsions can be approximated by

$$F_{ij}^r = \frac{2b}{(\epsilon + ||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(1-0)(y_i-y_j) = \frac{2b}{(\epsilon + ||y_i-y_j||^2)(1+a||y_i-y_j||^{2b})}(y_i-y_j).$$

Thus, in each epoch, we will be able to consider a significant number of attractions between points (possibly even all in certain epochs), but we will not be able to consider all repulsions for reasons of computational complexity. Next, we will describe one epoch of this algorithm. The learning rate will be chosen as the difference between 1 and the ratio between the current epoch and the total number of epochs. Then, we will select each pair (i,j) such that $p_{ij} \neq 0$. With probability p_{ij}, the vector y_i will be translated by the product of the learning rate and

$$\frac{F_{ij}^a}{p_{ij}} = -\frac{2ab||y_i-y_j||^{2b}}{(1+a||y_i-y_j||^{2b})(\epsilon_1+||y_i-y_j||^2)}(y_i-y_j).$$

Then, we will randomly choose $n_neg_samples$ (a hyperparameter) vectors y_c and for each one, we will translate y_i by the product of the learning rate and the repulsion exerted by y_c on y_i, approximating that $p_{ic} = 0$ every time. Because most of the time, the value p_{ic} will indeed be null, this approximation will not significantly change our final result, but it will save us the computational cost of accessing a random value in a sparse matrix. In the bibliographic reference [9], the following acceleration variant of the algorithm is proposed: every time a point exerts an attraction or repulsion on another point, the other will also exert a reaction, i.e., an equal and opposite force.
However, following the implementation of this algorithm, I observed that I was not obtaining good results at all (the clusters were not conserved upon dimensionality reduction), so I came up with a solution following several experiments. In each epoch, for each point y_i, we calculate the vectorial sum of all attractions and repulsions exerted on it, and if this norm exceeded $max_norm_rate \cdot \sqrt{d}$(which is the norm of a vector with all compennets equal to max_norm_rate), then we translate y_i with a vector of the same direction, but with a norm equal to the product of the learning rate and $max_norm_rate \cdot \sqrt{d}$. Otherwise, we translate it by the product of the learning rate and the respective sum (as in the initial recommendation).
We will attach a Python implementation of this variant of stochastic gradient descent in the subsection A.4.5 from the appendix.
Now, we will test it with a similar method as in the previous algorithm in the subsection A.4.6 from the appendix and we will attach the results.
As we can see, UMAP preserved the clusters, therefore validating its efficiency.



Appendix A

Appendix – Python Codes

A.1 SNE

A.1.1 Computing p_{ij} probabilities for SNE

```
import numpy as np


def squared_distances(x):
    x_norms = np.sum(x**2, axis=1)
    d = x_norms[:, np.newaxis] + x_norms.T[np.newaxis, :] - 2 * x @ x.T
    np.fill_diagonal(d, np.inf)
    return d
# The vector x_norms contains the squared norms of each x_i
# We compute the squared distances between each x_i and x_j using the
    formula:
# ||x_i - x_j||^2 = ||x_i||^2 + ||x_j||^2 - 2 * x_i * x_j, where * is the
     dot product

def p_SNE_formula(beta, d):
    p = np.exp(-beta * d)
    p /= np.sum(p)
    return p
# beta is beta_i, and p is a vector with p_{ij}, i fixed
# For a fixed i, computes p_{ij} based on beta_i using the formula
    derived earlier

def entropy(p):
    p_positive = p[p>0]
    return -p_positive @ (np.log2(p_positive))
# The vector p contains all probabilities p_{ij} for fixed i
# entropy(p) computes the entropy of the distribution {p_{ij} | j != i}

def p_SNE_value(k, k0, x, tol):
    # tol is the tolerance
    d = squared_distances(x)
    # Compute pairwise distances between points
    expected_H = np.log2(k0)
```

```
    # The target entropy value
    p = np.zeros((k, k))
    # Initialize the values p_{ij}
    for i in range(k):
      d_i = d[i, :]
      minn = np.min(d_i)
      index = np.where(np.isclose(d_i, minn))
      # Find the indices of points at the minimum distance from x_i
      a = len(index[0])
      # Count how many points are at minimum distance from x_i
      if a >= k0:
        p[i, index] = 1 / a
        # The other p_{ij} are already initialized to 0
      else:
        m = 1
        p_i = p_SNE_formula(m, d_i)
        deltaH = entropy(p_i) - expected_H
        while deltaH < tol:
          m /= 2
          p_i = p_SNE_formula(m, d_i)
          deltaH = entropy(p_i) - expected_H
          # We found m which is smaller than the correct beta
        if abs(deltaH) > tol:
          # If deltaH is already within tolerance, we found the correct
              p_{ij}
          M = 1
          p_i = p_SNE_formula(M, d_i)
          deltaH = entropy(p_i) - expected_H
          while deltaH > -tol:
            M *= 2
            p_i = p_SNE_formula(M, d_i)
            deltaH = entropy(p_i) - expected_H
            # We found M which is greater than the correct beta
          while abs(deltaH) > tol:
            beta = (m + M) / 2
            p_i = p_SNE_formula(beta, d_i)
            deltaH = entropy(p_i) - expected_H
            if deltaH > 0:
              m = beta
            else:
              M = beta
              #We performed binary search
        p[i, :] = p_i

    return p
```

A.1.2 Optimization in SNE

```
def q_SNE_formula(y):
  d=squared_distances(y)
  q=np.exp(-d)
  np.fill_diagonal(q,0)
```

```
  q/=np.sum(q,axis=1)[:, np.newaxis]
  return q
  # analogous to p_SNE_formula, but sigma_i=1
  # we calculate the whole matrix, not just one row

def gradient_SNE(psum,q,y):
  coef=psum-q-q.T
  # a matrix with p_{ij}-q_{ij}+p_{ji}-q_{ji}
  return y * coef.sum(axis=1)[:, np.newaxis] - coef @ y
  # y*coef.sum(axis=1)[:,np.newaxis] means y_i * sum of coef_{ij} on each
      row
  # coef@y means sum of coef_{ij} * y_j on each row
  # their difference is exactly half the gradient
  # we will use eta=0.2 instead of 0.1 to compensate

def objective_SNE(p_index,q,index):
   # the SNE objective function
  return -p_index@np.log(q.flatten()[index])
 # index represents the indices where p as a k^2*1 vector would be non-
     zero
  # p_index represents the non-zero p-values
  # in all positions where p is 0, q will also be 0 since only q_{ii}=0

def SNE(d,x,k0,tol,epochs):
  # d is the dimension of the reduced space
  # x are the points in the attribute space
  # epochs is the number of gradient descent epochs
  k=x.shape[0]
  p=p_SNE_value(k, k0, x, tol)
  psum=p+p.T
  p_vect=p.flatten()
  # p transformed into a vector
  index=np.where(p_vect!=0)
  # indices where it is not zero
  p_index=p_vect[index]
  # eliminate zero elements
  alpha=0.5
  # initialize alpha from the momentum formula
  y=np.random.normal(loc=0.0, scale=0.0001, size=(k, d))
  # initialize y
  y_prev=y.copy()
  # initialize previous y, as it appears in the formula
  obj_min=float('inf')
  # initialize the minimum objective function value with infinity
  for i in range(epochs):
    q=q_SNE_formula(y)
    obj=objective_SNE(p_index,q,index)
    if obj<obj_min:
      obj_min=obj
      y_min=y.copy()
    # count the minimum points among the explored points
    y_copy=y.copy()
    y+=(-0.2*gradient_SNE(psum,q,y))+alpha*(y-y_prev)
```

```
    # apply the formula from gradient descent with momentum
    y_prev=y_copy
    # save the previous y to reuse it in the formula
    if i>=250:
      alpha=0.8
    # update alpha according to the described rule
  return y_min
```

A.1.3 Testing SNE

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs # import make_blobs to generate
    clusters

# Generate test data with two clusters
# n_samples: total number of points
# centers: number of clusters
# n_features: dimension of the original space (number of attributes)
# cluster_std: standard deviation of the clusters
# x is a matrix with the x_i vectors on the rows
#x_cluster is boolean value for the cluster of each x_i
x, x_cluster = make_blobs(n_samples=200, centers=2, n_features=10,
                          cluster_std=1.5)

# Parameters for the SNE algorithm
d = 2       # dimension of the reduced space (2D for visualization)
k0 = 15
tol = 1e-5      # tolerance for calculating p_ij probabilities
epochs = 1000

# Run the SNE algorithm
y = SNE(d, x, k0, tol, epochs)



# Visualize SNE results
plt.figure(figsize=(8, 8))
# Use 'c=x_cluster' to color the points according to their original
    cluster label.
# 'cmap='viridis'' is a color palette.
# s is the size of each point
# alpha is the opacity of each point
plt.scatter(y[:, 0], y[:, 1], c=x_cluster, cmap='viridis', s=30, alpha
    =0.8)
plt.title('SNE - Cluster Visualization (2D)')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.grid(True)
plt.show()
```

A.2 Symmetric SNE

A.2.1 Computing p_{ij} probabilities in Symmetric SNE

```
import numpy as np


def squared_distances(x):
    x_norms = np.sum(x**2, axis=1)
    d = x_norms[:, np.newaxis] + x_norms.T[np.newaxis, :] - 2 * x @ x.T
    np.fill_diagonal(d, np.inf)
    return d

def p_SNE_formula(beta, d):
    p = np.exp(-beta * d)
    p /= np.sum(p)
    return p

def entropy(p):
    p_positive = p[p>0]
    return -p_positive @ (np.log2(p_positive))

def p_SNE_value(k, k0, x, tol):
    d = squared_distances(x)
    expected_H = np.log2(k0)
    p = np.zeros((k, k))
    for i in range(k):
      d_i = d[i, :]
      minn = np.min(d_i)
      index = np.where(np.isclose(d_i, minn))
      a = len(index[0])
      if a >= k0:
        p[i, index] = 1 / a
      else:
        m = 1
        p_i = p_SNE_formula(m, d_i)
        deltaH = entropy(p_i) - expected_H
        while deltaH < tol:
          m /= 2
          p_i = p_SNE_formula(m, d_i)
          deltaH = entropy(p_i) - expected_H
        if abs(deltaH) > tol:
          M = 1
          p_i = p_SNE_formula(M, d_i)
          deltaH = entropy(p_i) - expected_H
          while deltaH > -tol:
            M *= 2
            p_i = p_SNE_formula(M, d_i)
            deltaH = entropy(p_i) - expected_H
          while abs(deltaH) > tol:
            beta = (m + M) / 2
            p_i = p_SNE_formula(beta, d_i)
            deltaH = entropy(p_i) - expected_H
```

```
            if deltaH > 0:
              m = beta
            else:
              M = beta
        p[i, :] = p_i

    return p

def p_SymmetricSNE_value(k,k0,x,tol):
  # this additional function is the only difference compared to SNE in
      the computation of the p_{ij} values
  p_before=p_SNE_value(k,k0,x,tol)
  p=(p_before+p_before.T)/(2*k)
  # I have implemented the formula discussed in this chapter
  return p
```

A.2.2 Optimization in Symmetric SNE

```
def q_SymmetricSNE_formula(y):
  # here I used the formula for q_{ij} from Symmetric SNE
  d=squared_distances(y)
  q=np.exp(-d)
  np.fill_diagonal(q,0)
  q/=np.sum(q.flatten())
  return q

def gradient_SymmetricSNE(p,q,y):
  # here, I implemented the formula of the gradient of Symmetric SNE
  coef=p-q
  return y * coef.sum(axis=1)[:, np.newaxis] - coef @ y
  # their difference is exactly a quarter of the the gradient
  # we will use eta=0.4 instead of 0.1 to compensate

def objective_SNE(p_index,q,index):
  return -p_index@np.log(q.flatten()[index])

def SymmetricSNE(d,x,k0,tol,epochs):
  k=x.shape[0]
  p=p_SymmetricSNE_value(k, k0, x, tol)
  # here, I use the p_{ij} values from Symmetric SNE
  p_vect=p.flatten()
  index=np.where(p_vect!=0)
  p_index=p_vect[index]
  alpha=0.5
  y=np.random.normal(loc=0.0, scale=0.0001, size=(k, d))
  y_prev=y.copy()
  obj_min=float('inf')
  for i in range(epochs):
    q=q_SymmetricSNE_formula(y)
    # I use the q_{ij} values from Symmetric SNE
    obj=objective_SNE(p_index,q,index)
    if obj<obj_min:
```

```
      obj_min=obj
      y_min=y.copy()
    y_copy=y.copy()
    y+=(-0.4*gradient_SymmetricSNE(p,q,y))+alpha*(y-y_prev)
    # eta=0.4, since we divided the gradient by 4, not only 2
    # also, I used the gradient from Symmetric SNE
    y_prev=y_copy
    if i>=250:
      alpha=0.8
  return y_min
```

A.2.3 Testing Symmetric SNE

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

x, x_cluster = make_blobs(n_samples=200, centers=2, n_features=10,
                          cluster_std=1.5)

d = 2
k0 = 15
tol = 1e-5
epochs = 1000

# Run the Symmetric SNE algorithm
y = SymmetricSNE(d, x, k0, tol, epochs)



plt.figure(figsize=(8, 8))
plt.scatter(y[:, 0], y[:, 1], c=x_cluster, cmap='viridis', s=30, alpha
    =0.8)
plt.title('Symmetric SNE - Cluster Visualization (2D)')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.grid(True)
plt.show()
```

A.3 tSNE

A.3.1 Computing p_{ij} and q_{ij} probabilities in tSNE

```
import numpy as np


def squared_distances(x):
    x_norms = np.sum(x**2, axis=1)
    return x_norms[:, np.newaxis] + x_norms.T[np.newaxis, :] - 2 * x @ x.
        T
```

```
def p_SNE_formula(beta, d):
    p = np.exp(-beta * d)
    p /= np.sum(p)
    return p

def entropy(p):
    p_positive = p[p>0]
    return -p_positive @ (np.log2(p_positive))

def p_SNE_value(k, k0, x, tol):
    d = squared_distances(x)
    np.fill_diagonal(d, np.inf)
    expected_H = np.log2(k0)
    p = np.zeros((k, k))
    for i in range(k):
      d_i = d[i, :]
      minn = np.min(d_i)
      index = np.where(np.isclose(d_i, minn))
      a = len(index[0])
      if a >= k0:
        p[i, index] = 1 / a
      else:
        m = 1
        p_i = p_SNE_formula(m, d_i)
        deltaH = entropy(p_i) - expected_H
        while deltaH < tol:
          m /= 2
          p_i = p_SNE_formula(m, d_i)
          deltaH = entropy(p_i) - expected_H
        if abs(deltaH) > tol:
          M = 1
          p_i = p_SNE_formula(M, d_i)
          deltaH = entropy(p_i) - expected_H
          while deltaH > -tol:
            M *= 2
            p_i = p_SNE_formula(M, d_i)
            deltaH = entropy(p_i) - expected_H
          while abs(deltaH) > tol:
            beta = (m + M) / 2
            p_i = p_SNE_formula(beta, d_i)
            deltaH = entropy(p_i) - expected_H
            if deltaH > 0:
              m = beta
            else:
              M = beta
        p[i, :] = p_i

    return p

def p_SymmetricSNE_value(k,k0,x,tol):
  # this additional function is the only difference compared to SNE in
      the computation of the p_{ij} values
  p_before=p_SNE_value(k,k0,x,tol)
```

```
  p=(p_before+p_before.T)/(2*k)
  # I have implemented the formula discussed in this chapter
  return p

def q_tSNE_formula(y):
  # here I used the formula for q_{ij} from tSNE
  d=squared_distances(y)
  q=1/(1+d)
  np.fill_diagonal(q,0)
  q/=np.sum(q.flatten())
  return q
```

A.3.2 Optimization in tSNE

```
def gradient_tSNE(p,q,y,squared_dist):
  # here, I implemented the formula of the gradient of tSNE
  coef=(p-q)/(1+squared_dist)
  return y * coef.sum(axis=1)[:, np.newaxis] - coef @ y
  # their difference is exactly a quarter of the the gradient
  # we will use eta=0.4 instead of 0.1 to compensate

def objective_SNE(p_index,q,index):
  return -p_index@np.log(q.flatten()[index])

def tSNE(d,x,k0,tol,epochs):
  k=x.shape[0]
  p=p_SymmetricSNE_value(k, k0, x, tol)
  p_vect=p.flatten()
  index=np.where(p_vect!=0)
  p_index=p_vect[index]
  alpha=0.5
  y=np.random.normal(loc=0.0, scale=0.0001, size=(k, d))
  y_prev=y.copy()
  obj_min=float('inf')
  for i in range(epochs):
    squared_dist=squared_distances(y)
    q=q_tSNE_formula(squared_dist)
    # I use the q_{ij} values from tSNE
    obj=objective_SNE(p_index,q,index)
    if obj<obj_min:
      obj_min=obj
      y_min=y.copy()
    y_copy=y.copy()
    y+=(-0.4*gradient_tSNE(p,q,y,squared_dist))+alpha*(y-y_prev)
    # eta=0.4, since we divided the gradient by 4, not only 2
    # also, I used the gradient from tSNE
    y_prev=y_copy
    if i>=250:
      alpha=0.8
  return y_min
```

A.3.3 Testing tSNE

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

x, x_cluster = make_blobs(n_samples=200, centers=2, n_features=10,
                          cluster_std=1.5)

d = 2
k0 = 15
tol = 1e-5
epochs = 1000

# Run the tSNE algorithm
y = tSNE(d, x, k0, tol, epochs)



plt.figure(figsize=(8, 8))
plt.scatter(y[:, 0], y[:, 1], c=x_cluster, cmap='viridis', s=30, alpha
    =0.8)
plt.title('tSNE - Cluster Visualization (2D)')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.grid(True)
plt.show()
```

A.4 UMAP

A.4.1 UMAP and Nearest Neighbors Descent

```
import numpy as np
import pynndescent

def nn_descent(x,k0, max_iters):
  index = pynndescent.NNDescent(x, n_neighbors=k0+1, n_iters=max_iters,
      metric='euclidean')
  neighbors, distances = index.neighbor_graph
  distances=distances[:,1:]
  neighbors=neighbors[:,1:]
  rho = np.min(distances, axis=1, keepdims=True)
  distances = distances - rho
  return neighbors, distances
```

A.4.2 UMAP and Binary Search

```
import numpy as np
from numba import njit, prange
from scipy.sparse import csr_matrix
```

```
@njit(parallel=True, fastmath=True)
def p_weights(data, indptr, k, k0, expected, tol):
    out = np.empty(data.shape[0], dtype=np.float64)

    for i in prange(k):
        s = indptr[i]

        m = 1.0
        sum_temp = 0.0
        for t in range(k0):
            sum_temp += np.exp(-data[s + t] / m)
        delta = sum_temp - expected


        while delta > tol:
            m /= 2.0
            sum_temp = 0.0
            for t in range(k0):
                sum_temp += np.exp(-data[s + t] / m)
            delta = sum_temp - expected


        if abs(delta) > tol:
            M = 1.0
            sum_temp = 0.0
            for t in range(k0):
                sum_temp += np.exp(-data[s + t] / M)
            delta = sum_temp - expected


            while delta < -tol:
                M *= 2.0
                sum_temp = 0.0
                for t in range(k0):
                    sum_temp += np.exp(-data[s + t] / M)
                delta = sum_temp - expected


            while abs(delta) > tol:
                sigma = (m + M) / 2.0
                sum_temp = 0.0
                for t in range(k0):
                    sum_temp += np.exp(-data[s + t] / sigma)

                delta = sum_temp - expected
                if delta > 0:
                    M = sigma
                else:
                    m = sigma

            sigma = (m + M) / 2.0
        else:
            sigma = m
```

```
        for t in range(k0):
            out[s + t] = np.exp(-data[s + t] / sigma)

    return out

def p_UMAP(x, k, k0, tol, max_iters):

    neighbors, distances = nn_descent(x, k0, max_iters)


    indptr = np.arange(0, k*k0 + 1, k0)
    data = distances.ravel()

    expected = np.log2(k0)
    p_data = p_weights(data, indptr, k, k0, expected, tol)

    p = csr_matrix((p_data, neighbors.ravel(), indptr), shape=(k, k))
    pt = p.T
    p = p + pt - p.multiply(pt)
    return p
```

A.4.3 UMAP and Laplacian Eigenmaps

```
from scipy.sparse import diags, identity
from scipy.sparse.csgraph import connected_components
from scipy.sparse.linalg import eigsh

def Laplacian_eigenmap(k, p, d):

    D = p.sum(axis=1).A1
    D_inv_sqrt = diags(1 / np.sqrt(D))
    I = identity(k)
    L_tilde = I - D_inv_sqrt @ p @ D_inv_sqrt
    #we calculated L_tilde as a sparse matrix

    m, _ = connected_components(p, directed=False)
    #m is the number of connected components
    nr_vec=m+d
    #total number of calculated eignevector

    vals, vecs = eigsh(L_tilde, min(nr_vec,k), which='SM')
    y = vecs[:,m:]
    while y.shape[1] < d:
        y = np.concatenate([y, y], axis=1)
    y = y[:, :d]

    return y
```

A.4.4 UMAP and Non-Linear Least Squares

```
from scipy.integrate import quad
from scipy.optimize import minimize

def NLLS_UMAP(d_min, epsilon):

    def psi(x):
        return 1 if x <= d_min else np.exp(-(x - d_min))

    def phi(x, a, b):
        return 1 / (1 + a * np.power(x, 2 * b))

    def integrand(x, a, b):
        return (phi(x, a, b) - psi(x)) ** 2

    def objective(params):
        a, b = params
        if a < epsilon or b <= 0.25:
            return np.inf
        integral, _ = quad(integrand, 0, np.inf, args=(a, b))
        return integral

    result = minimize(objective, [1, 0.5], method='L-BFGS-B', bounds=[(
        epsilon, None), (0.25, None)])
    a, b = result.x
    if np.isnan(a) or np.isinf(a) or a < epsilon:
      a = epsilon
    if np.isnan(b) or np.isinf(b) or b < 0.25:
      b = 0.25
    return a, b
```

A.4.5 UMAP and Stochastic Gradient Descent

```
from math import sqrt

@njit(parallel=False)
def compute_gradient(y, p_data, p_indices, p_indptr, a, b, n_neg_sample,
    k, d, epsilon1, epsilon2, max_norm, grad):
    grad.fill(0)
    c_rep = 2.0 * b
    c_at = -c_rep * a


    for i in range(k):
        diff = np.zeros(d)


        for pj in range(p_indptr[i], p_indptr[i + 1]):
            j = p_indices[pj]
            pij = p_data[pj]
```

```
            if np.random.random() < pij:
                dist2 = 0.0
                for t in range(d):
                    val = y[i, t] - y[j, t]
                    diff[t] = val
                    dist2 += val * val

                dist2b = dist2 ** b
                coeff = (c_at * dist2b) / ((dist2 + epsilon1) * (1.0 + a
                    * dist2b))

                for t in range(d):
                    g = coeff * diff[t]
                    grad[i, t] += g
                    grad[j, t] -= g


        for _ in range(n_neg_sample):
            c = np.random.randint(0, k - 1)

            dist2 = 0.0
            for t in range(d):
                val = y[i, t] - y[c, t]
                diff[t] = val
                dist2 += val * val

            coeff = c_rep / ((epsilon2 + dist2) * (1.0 + a * dist2 ** b))

            for t in range(d):
                g = coeff * diff[t]
                grad[i, t] += g
                grad[c, t] -= g

    for i in range(k):
        s = 0.0
        for t in range(d):
            s += grad[i, t] * grad[i, t]
        nrm = sqrt(s)
        if nrm > max_norm:
            scale = max_norm / (nrm + 1e-12)
            for t in range(d):
                grad[i, t] *= scale

    return grad
```

A.4.6 Testing UMAP

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

x, x_cluster = make_blobs(n_samples=200, centers=2, n_features=10,
                          cluster_std=1.5)
```

```
# Run the UMAP algorithm
y = UMAP(
    k=x.shape[0],
    k0=15,
    d=2,
    x=x,
    tol=1e-5,
    max_iters=10,
    d_min=0.1,
    epochs=300,
    n_neg_sample=10,
    epsilon1=1e-8,
    epsilon2=1e-8,
    epsilon=1e-8,
    max_norm_rate=4
)



plt.figure(figsize=(8, 8))
plt.scatter(y[:, 0], y[:, 1], c=x_cluster, cmap='viridis', s=30, alpha
    =0.8)
plt.title('UMAP - Cluster Visualization (2D)')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.grid(True)
plt.show()
```

Bibliography

[1] Beyer, K., Goldstein, J., Ramakrishnan, R., Shaft, U. *When is "Nearest Neighbor" Meaningful?*, Database Theory — ICDT'99, Lecture Notes in Computer Science, vol. 1540, Springer, 1999, pp. 217–235. doi:10.1007/3-540-49257-7_15. ISBN 978-3-540-65452-0.

[2] Pestov, Vladimir. *On the Geometry of Similarity Search: Dimensionality Curse and Concentration of Measure.* Lecture Notes in Computer Science, vol. 2081, Springer, 2001, pp. 42–55. doi:10.1007/3-540-44794-6_4.

[3] Hinton, G. E., & Roweis, S. T. *Stochastic Neighbor Embedding.* In S. Becker, S. Thrun, & K. Obermayer (Eds.), *Advances in Neural Information Processing Systems 15* (pp. 857–864). MIT Press, 2003.

[4] B. Ghojogh, M. Crowley, F. Karray, and A. Ghodsi, *"Stochastic Neighbour Embedding"*, in *Elements of Dimensionality Reduction and Manifold Learning*, Springer, 2023, pp. 455–476. doi: 10.1007/978-3-031-10602-6_16.

[5] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.

[6] L. McInnes, J. Healy, and J. Melville, *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*, arXiv preprint arXiv:1802.03426, 2018.

[7] Wei Dong, Moses Charikar, and Kai Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web (WWW '11)*, pages 577–586, ACM, 2011. DOI: 10.1145/1963405.1963487.

[8] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003.

[9] B. Ghojogh, F. Karray, M. Crowley, and A. Ghodsi, *"Uniform Manifold Approximation and Projection (UMAP)"*, in *Elements of Dimensionality Reduction and Manifold Learning*, Springer, 2023, pp. 477–503. doi: 10.1007/978-3-031-10602-6_17